AetherProof: Autonomous Formal Verification and Invariant Synthesis for Multi-Agent Software Ecosystems

A Formal Architecture for Specification-Driven Proof Construction and Counterexample-Guided Remediation

Samir Baladi

Interdisciplinary AI Researcher — Ronin Institute / Rite of Renaissance

Research Article — Formal Methods / Software Engineering / Distributed Systems
Status: Formal Architecture Proposal with Illustrative Analysis — No Implementation Benchmarked
DOI: https://doi.org/10.5281/zenodo.24013124
ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com
Record ID: AETHERPROOF-01 · Version 1.0.0 — November 2026

Abstract

Multi-agent software ecosystems generate changesets at a rate at which continuous-integration testing, even when performed after merge as in AetherFlow, can detect many failures but cannot establish structural correctness. This paper presents AetherProof, a formal architecture for specification-driven proof construction, incremental differential proving, and counterexample-guided remediation. It makes three contributions. First, a three-layer *functional* specification architecture — TLA+ for system properties, Why3 and Lean for functional contracts, SMT-LIB with Z3 for logical constraints — that is explicitly not hierarchical: the layers coexist and are related by explicit semantic relations rather than unified into one language. Second, a Canonical Specification Representation (CSR) whose records carry three distinct identities (record, artifact, semantic) and are linked by seven typed relations and a two-axis translation classification. Third, a Proof Dependency Graph (PDG) that extends an obligation dependency graph with evidence and assumption nodes, supports incremental invalidation and conditional reuse of proof evidence, and drives cost-aware, tiered proving. The paper further specifies a bounded counterexample-guided remediation cycle with guardrails against specification gaming, a message-based integration protocol with AetherFlow, a human review protocol with separation of duties, and a catalogue of fifteen threats with mitigations and residual risk. Ten propositions are stated as design requirements and are not formally proven. No implementation is built and no empirical evaluation is performed; this is a formalization-stage contribution. Its central commitment is that the hard problem is not making TLA+, Why3, Lean, and SMT-LIB exchange data, but establishing that what these languages mean remains consistent when a specification moves between representations — an open question (Q2) that this paper frames but does not solve.

Keywords: *Formal Verification; Specification Languages; TLA+; Why3; Lean; SMT; Proof Obligations; Incremental Proving; Counterexample-Guided Remediation; Agentic Software Engineering; AetherProof.*

Evidentiary status: *AetherProof is a proposed formal architecture. No implementation has been built, no benchmark executed, and no empirical evaluation performed.* · ***Propositions P1–P10*** *are design requirements, not theorems.* · ***Numerical values*** *appear only as parameters of a hypothetical illustration or as defaults of the design (for example, a remediation iteration bound of 3); none is a measurement.* · *Descriptions of third-party tools and systems follow the cited public sources and were not independently verified.*

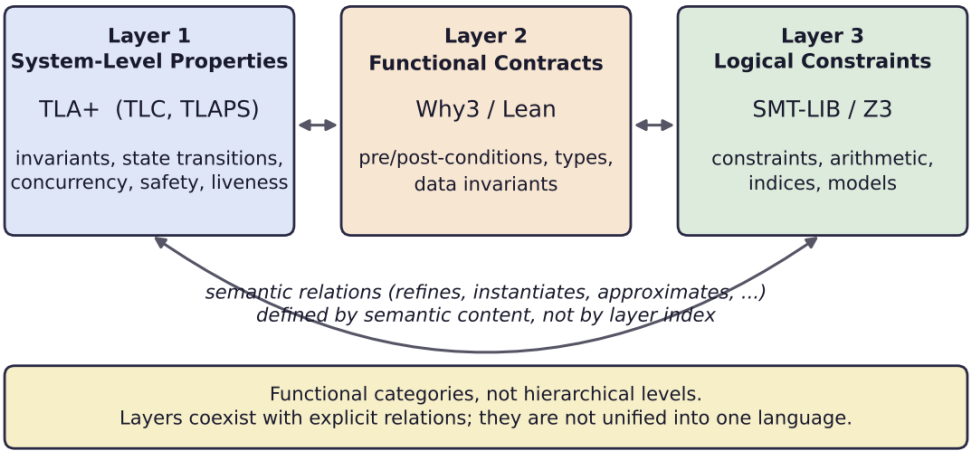


Figure 1 (Fig1_Three_Layer_Architecture.png). The three-layer functional specification architecture. Double-headed arrows denote semantic relations, not refinement; the layers are functional categories, not hierarchical levels.

Table of Contents

1. Introduction	3
2. Background and Related Work	5
3. Formal Framework	7
4. Proof Obligations	12
5. Incremental Differential Proof Engine	16
6. Counterexample-Guided Remediation	21
7. Integration with AetherFlow	24
8. Human Review Protocol	27
9. Threats and Limitations	31
10. System Architecture	33
11. Discussion	37
12. Conclusion and Future Work	38
Appendix A — Notation	40
Appendix B — Open Questions Q1–Q50	40
Appendix C — Design Decisions D1–D52	42
Appendix D — Threats T1–T15	43
References	44
Research Status Statement	46

1. Introduction

1.1 The Verification Bottleneck at Agent Scale

Software verification has traditionally been performed by two broad families of techniques. Testing-based techniques, embodied in continuous integration (CI), execute the system on selected inputs and report failures. Proof-based techniques, from Hoare-style deductive verification (Hoare, 1969) to interactive theorem proving and model checking, establish that a stated property holds for *all* behaviors admitted by a model, under stated assumptions. The two are complementary: tests find defects cheaply but cannot show their absence, while proofs can show absence relative to a specification but require a specification to exist and be correct.

Autonomous coding agents alter the economics of both. A fleet of agents can produce changesets continuously and in parallel, at a volume that exceeds the capacity of human reviewers to read each change and of test suites to exercise each interaction. Recent architectural work on the integration layer, notably AetherFlow (Baladi, 2026c), responds by admitting changesets without waiting for verification, verifying them asynchronously after admission, and repairing failures with remediation agents. That design makes *detection* of failures timely, but detection through CI has a structural ceiling: a passing test suite is evidence about the tested inputs, not about the system's invariants. When thousands of changesets modify interacting components, the question that matters — whether a system-level invariant such as “the work queue never exceeds its bound” still holds — is not answered by a green build.

This paper asks what architecture would allow proof-based verification to operate at the same cadence as agent-generated change. Three obstacles are apparent. First, **heterogeneity**: different properties are naturally expressed in different formalisms (temporal properties of a concurrent protocol in TLA+, function contracts in a deductive verifier, arithmetic constraints in SMT), and no single language serves all. Second, **incrementality**: re-proving everything after every change is prohibitively expensive, yet reusing a stale proof is unsound. Third, **trust**: an autonomous system that generates specifications, proofs, and patches can satisfy a verifier by weakening it, so the architecture must make weakening visible and attributable.

1.2 Relationship to AetherGit and AetherFlow

AetherProof is the third layer of a three-layer stack. **AetherGit** (Baladi, 2026b) is a storage-layer architecture: an append-only changeset log in which each changeset is a tuple $c_i = (\delta_i, P(c_i), \tau_i, \sigma_i)$ of content delta, parent set, timestamp, and content-addressed signature. **AetherFlow** (Baladi, 2026c) is an integration-layer architecture: lock-free admission to the trunk log, a recursive three-valued verification operator (verified, failed, tainted), and graded remediation. Both are formal architecture papers; neither is relied upon here for any empirical claim. Both descend from the author's structural analysis (Baladi, 2026a) of Anthropic's published account of its Scion source-control system (Anthropic, 2026).

DOI: https://doi.org/10.5281/zenodo.24013124

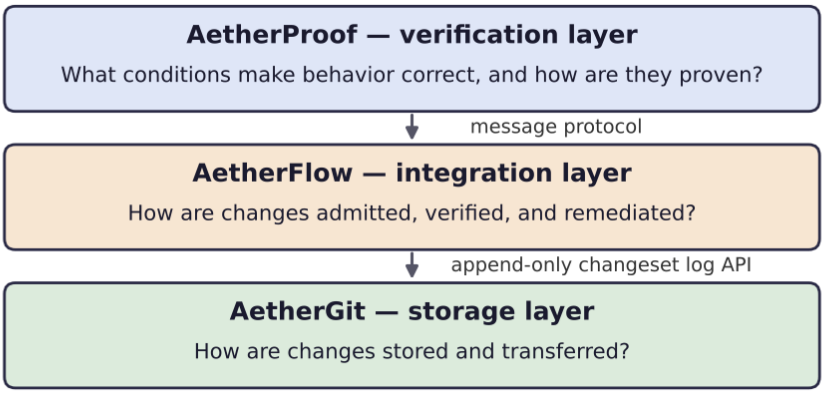


Figure 2. Position of AetherProof in the Aether family. Integration between layers is by message protocol, not by shared code.

The three layers are **methodologically independent**. AetherGit requires only an append-only log; AetherFlow requires only a changeset log with parent pointers; AetherProof requires only a CSR record format and a source of changesets. No layer is a precondition for another's formal claims. AetherProof is self-contained: a reader who has not seen the other two papers needs only the definitions in Section 3 and the protocol summary in Section 7. In AetherFlow's terms, AetherProof can be read as a producer of one specialized kind of evidence for the CI(c) term of the verification operator, namely proof evidence about specifications rather than test results about executions.

1.3 Contributions

1. A **three-layer functional specification architecture** (TLA+; Why3 and Lean; SMT-LIB and Z3) that is explicitly *not* hierarchical, together with a non-substitutability principle (Section 3.1).
2. A **Canonical Specification Representation (CSR)** with three distinct identities — record, artifact, and semantic — and explicit, typed semantic relations (Sections 3.2–3.3).
3. A **two-axis classification of translations** between representations, separating the semantic relationship from the verification property, and a provisional, explicitly open definition of semantic consistency (Sections 3.4–3.5).
4. A **proof obligation model** with explicit assumptions, an obligation dependency graph, an assumption-tracking discipline, and a discharge protocol with per-path trusted-computing-base (TCB) manifests (Section 4).
5. A **Proof Dependency Graph (PDG)** that supports incremental invalidation, conditional evidence reuse, and cost-aware tiered proving (Section 5).
6. A **bounded counterexample-guided remediation protocol** with a diagnosis procedure and guardrails against specification gaming (Section 6), a **message-based integration protocol** (Section 7), a **human review protocol** with separation of duties and a trust hierarchy (Section 8), and a **threat catalogue** (Section 9).
7. A **research agenda** of fifty open questions (Q1–Q50), fifty-two recorded design decisions (D1–D52), and ten stated propositions (P1–P10) (Appendices B–C and Section 3).

1.4 Evidentiary Status

This is a formalization-stage paper. **No implementation exists**: none of the components of Section 10 has been built. **No benchmark or empirical evaluation has been performed**: nothing in this paper reports proof time, throughput, scalability, usability, or defect-detection rate. **The propositions are design requirements**: where this paper gives a short justification, it argues internal consistency of the definitions, not a machine-checked or peer-verified theorem. Numerical values appear only as default parameters of the design (for example, a remediation iteration bound of three) or as hypothetical inputs to the illustration in Section 10.6, and are labeled as such. The paper deliberately does not claim to solve semantic consistency between specification languages (Q2); it claims only to give the problem a precise place in an architecture.

The decision to specify before implementing is deliberate and ordered: specify, validate, implement, evaluate. Section 12 sets out what each subsequent stage would require.

> **Limitations of this section.** This introduction describes the motivation and scope of a design; it does not establish that proof-based verification is feasible at the cadence of agent-generated change. That is an empirical question (Q26).

2. Background and Related Work

2.1 Formal Methods and Specification Languages

TLA+. The Temporal Logic of Actions (Lamport, 1994) describes a system as a set of behaviors — infinite sequences of states — constrained by an initial predicate and a next-state relation, and expresses safety and liveness properties in temporal logic. TLA+ is the specification language built on it (Lamport, 2002). Two tools discharge TLA+ properties by different means: the TLC model checker explores the state space of a finite instance, and the TLA+ Proof System (TLAPS) checks hierarchical proofs by delegating leaf obligations to backend provers (Chaudhuri et al., 2010). Industrial experience shows that lightweight use of such specifications can find subtle design errors in distributed systems (Newcombe et al., 2015). The two discharge modes differ in what they establish: TLC results are bounded by the instance explored, while TLAPS proofs are unbounded but depend on backend provers. AetherProof records this difference in the TCB manifest rather than collapsing it.

Deductive verification and contracts. Hoare logic (Hoare, 1969) gives the foundation for reasoning about programs through pre- and post-conditions. Platforms such as Why3 (Filliâtre & Paskevich, 2013) generate verification conditions from annotated programs and dispatch them to external provers through configurable transformations and drivers; Dafny (Leino, 2010) integrates a similar workflow into a language. The existence of a transformation pipeline between the annotated program and the final prover query is exactly why Why3's TCB is larger than the prover alone.

Why several languages. A recurring observation in industrial and research practice is that no single formalism is best for system-level concurrency, program-level contracts, and arithmetic reasoning at once. The usual response is to apply each formalism where it fits and relate the results informally. AetherProof's position is that the relation should itself be a recorded, typed, reviewable artifact.

2.2 Proof Assistants and SMT Solvers

Proof assistants. Lean 4 (de Moura & Ullrich, 2021) is a proof assistant and programming language whose proofs are terms checked by a small kernel. This *de Bruijn criterion* — a large, possibly untrusted proof-producing apparatus checked by a small trusted checker — is the reason proof terms are attractive as evidence: they can be re-checked independently of how they were produced. Large verification efforts have used this principle at scale: seL4's functional-correctness proof (Klein et al., 2009; Klein et al., 2014) and the CompCert verified compiler (Leroy, 2009) are the canonical examples. Both also illustrate an important caveat that AetherProof inherits: a proof is relative to a specification and to a trusted base, and the specification may fail to capture intent.

SMT solvers. Satisfiability modulo theories solvers such as Z3 (de Moura & Bjørner, 2008) decide satisfiability of formulas over combinations of theories (arithmetic, arrays, bit-vectors). SMT-LIB (Barrett et al., 2010) standardizes the input language. An `unsat` answer can establish a property by showing its negation unsatisfiable, but the answer is only as trustworthy as the solver, unless the solver emits a proof object that an independent checker validates. For this reason AetherProof treats an `unsat` without a proof artifact as weaker evidence than a checked one (Section 4.5, Section 8.8).

Trusting the toolchain. The classic argument of Thompson (1984) — that trust in a program ultimately depends on every tool used to produce it — motivates making the trusted computing base an explicit artifact. AetherProof does not claim to eliminate the TCB; it requires each verification path to declare it.

2.3 Multi-Agent Software Engineering

Evaluation of language-model agents on real repository tasks has become a standard way to measure agent capability (Jimenez et al., 2024), and automated program repair has a longer lineage (Le Goues et al., 2012). At the infrastructure level, large monorepo systems (Potvin & Levenberg, 2016; Sapling SCM, n.d.) and the disclosed Scion architecture (Anthropic, 2026) show how storage and integration scale. Merge-queue research (Ananthanarayanan et al., 2019) addresses keeping a mainline green at scale. Flaky tests, which complicate any failure-attribution scheme, have been studied empirically (Luo et al., 2014). Within the Aether family, AetherGit and AetherFlow address storage and integration; what has been missing is a layer that asks whether the system's *stated* invariants hold, with machine-checkable evidence.

2.4 Counterexample-Guided Abstraction Refinement (CEGAR)

CEGAR (Clarke et al., 2000) verifies a property on an abstraction of the system; if the abstraction yields a counterexample, the counterexample is checked against the concrete system, and if it is spurious, the abstraction is refined to exclude it. The loop is the closest established ancestor of AetherProof's remediation cycle, and the abstract-interpretation view (Cousot & Cousot, 1977) supplies the vocabulary of over- and under-approximation used in Axis A of Section 3.4.

AetherProof differs from CEGAR in three respects, which is also why it cannot simply reuse CEGAR's guarantees. First, CEGAR refines an *abstraction* to remove spurious counterexamples; AetherProof must additionally decide whether a genuine counterexample indicates a code defect, a specification error, or a violated assumption, and treats these differently. Second, CEGAR terminates under conditions on the abstraction domain; AetherProof's cycle is instead *bounded by policy* and escalates to a human when the bound is reached. Third, CEGAR's refinement acts on the model; AetherProof's remediation may act on code, specifications, or assumptions, and the latter two are privileged operations subject to review.

DOI: https://doi.org/10.5281/zenodo.24013124

2.5 Research Gap

Existing work provides mature verifiers for individual formalisms, scalable infrastructure for large repositories, and agentic repair. What is not provided, to the author's knowledge, is a single architecture that (a) records, as reviewable artifacts, the typed relations between specifications in *different* formalisms and the assumptions each depends on; (b) maintains a dependency structure over obligations, evidence, and assumptions that supports sound incremental reuse as a changeset DAG evolves; and (c) specifies guardrails for an autonomous remediator that is otherwise able to satisfy a verifier by editing the verifier's inputs. AetherProof is a proposal to fill that gap at the level of architecture and specification, not of implementation.

Limitations of this section. The survey in this section is selective, not systematic, and the claim of a gap is made relative to the sources consulted. Statements about third-party tools follow the cited publications; versions and behaviors may have changed. The comparison with CEGAR is structural and does not claim that AetherProof inherits any of CEGAR's formal guarantees.

3. Formal Framework

This section defines the specification-level objects of AetherProof: the three functional layers, the CSR record, the semantic relations among records, the two-axis classification of translations, and a provisional notion of semantic consistency. Definitions are normative for the architecture; propositions are design requirements, each accompanied where possible by a short justification of internal consistency.

3.1 Three-Layer Functional Specification Architecture

AetherProof partitions specification content into three *functional* layers according to the kind of statement being made and the verification mode that suits it (Figure 1).

Layer	Language family	Functional scope	Verification modes
L1 — System-level properties	TLA+	System invariants, state transitions, concurrency, safety, liveness	Model checking (TLC); proof (TLAPS)
L2 — Functional contracts	Why3, Lean	Function contracts, pre/post-conditions, types, data invariants	Deductive verification (Why3); proof construction (Lean)
L3 — Logical constraints	SMT-LIB	Logical constraints, arithmetic, indices, models	SMT solving (e.g., Z3)

Layer 1 — system-level properties. L1 states what must hold of the system as a whole across its reachable states: an invariant (“the queue length never exceeds MAX”), a transition constraint, or a liveness property. Its natural object is a state machine and its natural verifier is a model checker or a proof over behaviors.

Layer 2 — functional contracts. L2 states what individual functions and data types must satisfy: preconditions, postconditions, type constraints, and data-structure invariants. Its natural object is a program or an abstract functional model, and its natural verifier is a deductive verifier or a proof assistant.

Layer 3 — logical constraints. L3 states quantifier-light arithmetic and logical facts — index bounds, linear arithmetic over counters, bit-level conditions — that arise as verification conditions or as stand-alone models. Its natural verifier is an SMT solver.

3.1.1 Non-Substitutability Principle

The layers are **not interchangeable**. The architecture assumes *coexistence with explicit semantic relations*, not *unification into one language*. Three reasons motivate the principle. **Expressiveness:** temporal properties over infinite behaviors, including liveness, have no direct counterpart in a quantifier-light constraint language without an encoding that itself must be justified. **Verification mode:** a bounded model-checking run, a kernel-checked proof term, and an SMT `unsat` answer are different kinds of evidence with different trusted bases. **Granularity:** a system invariant and the arithmetic fact that supports it are statements at different levels of abstraction, and conflating them hides the translation step at which meaning can be lost.

Proposition P1 (non-substitutability). *(design requirement; not formally proven).* The functional categories L1, L2, and L3 are non-substitutable: replacing the specification language assigned to one category by that of another is not assumed to preserve the meaning or the evidential strength of the statements assigned to it. *Justification sketch:* the three categories differ in the kind of object (behaviors, programs, formulas) and in the evidence modes available (Section 4.5); a replacement would require a translation, and by Definition 3 every translation must be classified rather than assumed exact.

3.1.2 Layers Are Functional, Not Hierarchical

It is tempting to read the three layers as a refinement chain from abstract (L1) to concrete (L3). AetherProof does not. Relations among specification elements are determined by their **semantic content**, not by layer index. An L3 constraint may approximate an L1 property; an L2 Lean lemma may instantiate a generic L3-style arithmetic fact; two L2 contracts in different languages may relate to each other directly. Figure 3 shows a typical requirement-driven trace in which refinement runs from L1 through L2 to L3, but this is one pattern among several and is a property of the recorded relations, not of the layers.

3.2 Canonical Specification Representation (CSR)

Definition 1 (CSR record). A CSR record is a tuple

```
csr = (csr_id, artifact_hash, semantic_id, layer, language, content,
       semantic_role, requirements_refs, assumptions, relations,
       version, provenance)
```

Field	Type	Meaning
csr_id	Identifier	Stable identifier of the record across its versions (record identity)
artifact_hash	Hash	Content hash of the concrete artifact (artifact identity)
semantic_id	Identifier	Identifier for the meaning and assumptions of the specification (semantic identity)
layer, language	L1/L2/L3; language tag	Functional layer and concrete language (e.g., TLA+, Why3, Lean, SMT-LIB)
content	Artifact	The specification text or structure itself
semantic_role	Enum	invariant (L1), contract (L2), constraint (L3), translation
requirements_refs	Set<RequirementID>	Requirements this record traces to; or marked unformalized
assumptions	Set<AssumptionID>	Assumptions under which the specification is intended to hold
relations	Set<SemanticRelation>	Typed relations to other CSR records (Section 3.3)
version, provenance	Version; Provenance	Version of the record; who or what produced it, with tool and timestamp

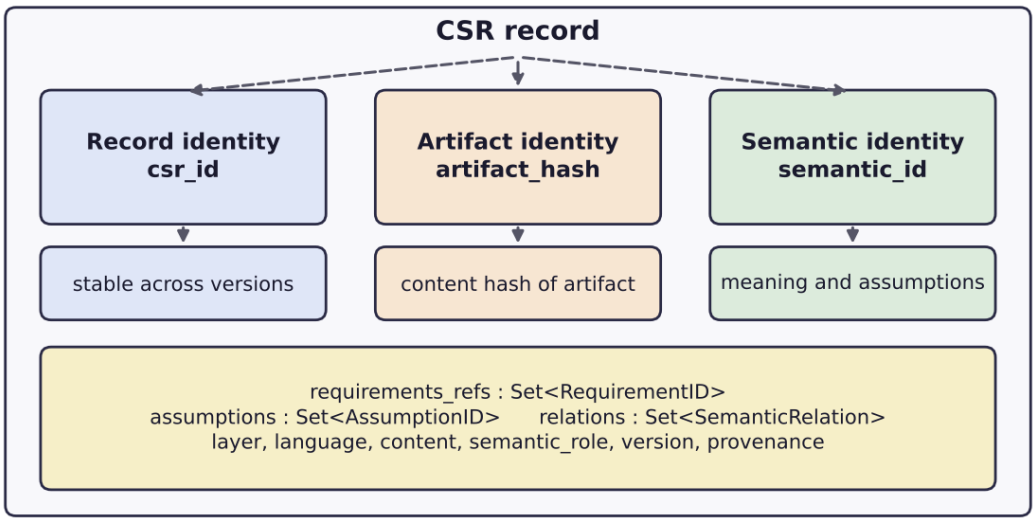


Figure 4 (Fig2_CSR_Record_Structure.png). CSR record structure: three identities and the relationship fields.

3.2.1 Three Identities

A single identifier cannot serve every purpose that a specification record has. AetherProof therefore separates three identities (design decision D7). The **record identity** (`csr_id`) is stable across versions and answers "which specification element is this?". The **artifact identity** (`artifact_hash`) is a content hash and answers "which exact bytes are these?". The **semantic identity** (`semantic_id`) is intended to answer "what does this mean, under which assumptions?". The intended behavior under edits is summarized below.

Edit	csr_id	artifact_hash	semantic_id
Reformatting; renaming a bound variable	unchanged	changes	intended to be unchanged
Strengthening or weakening the stated property	unchanged (new version)	changes	changes
Changing an assumption	unchanged (new version)	may change	changes
Re-expressing the same meaning in another language	new record (linked by a translation)	new	intended to be the same or explicitly related

The entries for `semantic_id` are *design intent*, not a guarantee: whether semantic identity can be computed or checked mechanically, and whether the three identities can be reconciled mechanically, are open questions (Q3 and Q7). Until they are resolved, assigning and comparing semantic identities is a reviewed, not an automated, act.

3.2.2 The CSR Is Not a Proof

The CSR **records** what specifications exist, what they structurally mean, and how they relate. It does **not** assert that any relation holds. A record stating that specification X refines specification Y is a claim, with provenance; whether the claim is true is a separate matter, addressed by obligations (Section 4) and review (Section 8). This separation is the central protection against a CSR being mistaken for evidence.

3.3 Semantic Relations

Definition 2 (semantic relation). A semantic relation between two CSR records is one of the seven typed relations below. Relations are recorded in the `relations` field and are independent of layer ordering.

Relation	Meaning	Direction
refines	Source refines target	Source → Target

Relation	Meaning	Direction
instantiates	Concrete instance of a generic specification	Instance → Generic
approximates	Source approximates target (with documented loss)	Direction declared
implies	Logical consequence	Within one layer
contradicts	Explicitly inconsistent	Symmetric
depends_on	Proof-level dependency	Dependent → Dependency
supersedes	Version replacement	New → Old

3.3.1 Well-Formedness Rules

1 **No circular refinement.** The relation refines must form a directed acyclic graph. Acyclicity of a finite relation graph is decidable by standard topological-sort in time linear in records plus relations, so this rule can be checked at record-insertion time.

2 **Explicit approximation.** Every approximates relation must carry a documented loss_description. An approximation without a stated loss is rejected as ill-formed.

3 **Contradiction recording.** A contradicts relation must be recorded as a **verification finding**.

3.3.2 Contradiction Is a Finding, Not an Escalation

When two records are recorded as contradictory, the architecture records a verification finding (design decision D8). It does not automatically escalate. The distinction matters: a contradiction is a statement about the specification set, and may be resolved by review of the specifications, by recognizing that two records have different scope or assumptions, or by retiring one via `supersedes`. Escalation to a human is triggered by separate policies (Section 8.3) such as conflicting evidence or an unresolved counterexample.

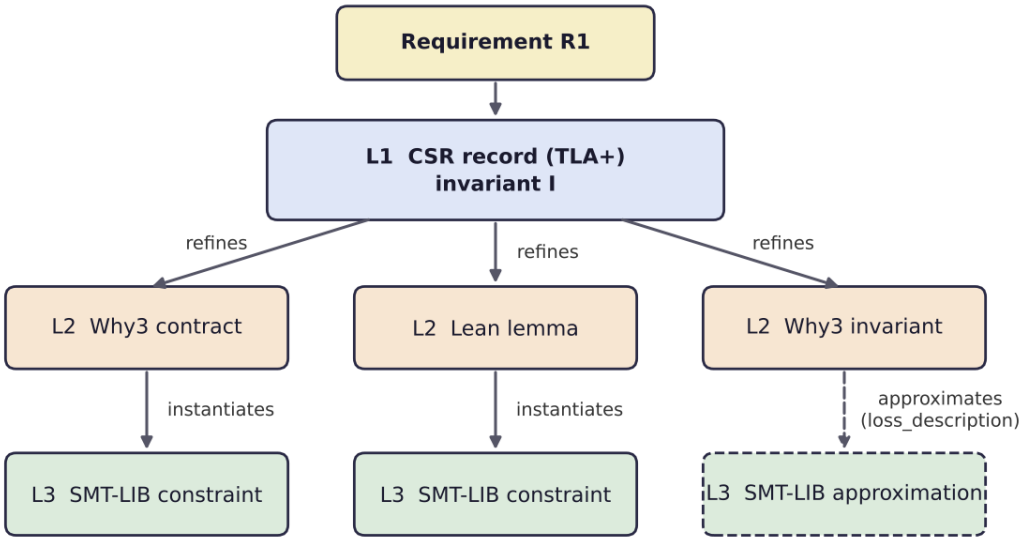


Figure 5 (Fig3_Semantic_Relations_Graph.png). A requirement-driven trace. Solid arrows: refines / instantiates. Dashed arrow: approximates, which carries a loss_description.

3.4 Two-Axis Translation Classification

A translation moves a specification from one representation to another (for example, from a TLA+ invariant to a Why3 contract, or from a contract to an SMT constraint). The central difficulty is that a translation can fail in two *different* ways: it can change what is meant, and it can fail to preserve a property of interest. AetherProof classifies these separately.

Definition 3 (translation classification). Let S be the source and T the target of a translation, and let B(S) and B(T) denote the sets of behaviors or models they admit. A translation is classified by two orthogonal axes.

Axis A — semantic relationship. *Exact*: B(S) and B(T) coincide under the translation. *Over-approximation*: B(T) admits at least the behaviors of B(S). *Under-approximation*: B(T) admits at most the behaviors of B(S). *Partial mapping*: the translation is defined only on part of the source. *Divergent*: the target does not stand in a recognized containment relation to the source.

Axis B — verification property, relative to a named property ϕ. *Sound* (w.r.t. ϕ): if the translated property is established on T, then ϕ holds on S. *Complete* (w.r.t. ϕ): if ϕ holds on S, then the translated property is establishable on T, so a refutation on T is a refutation on S. *Neither*: the translation is known to be neither sound nor complete for ϕ. *Not established*: no determination has been made.

Axis B — verification property (relative to a named property)

Axis A — semantic relationship	Sound	Complete	Neither	Not established
Exact	yes	yes	yes	yes
Over-approximation	yes	no	yes	yes
Under-approximation	no	yes	yes	yes
Partial mapping	depends	depends	yes	yes
Divergent	no	no	yes	yes

yes = compatible by definition no = incompatible by definition depends = depends on the specific mapping
Read for safety properties; direction of soundness may differ for other property classes.

Figure 6 (Fig4_Two_Axis_Classification.png). Compatibility of Axis A and Axis B values. yes = compatible by definition; no = incompatible by definition; depends = depends on the specific mapping.

Reading the matrix. For a *safety* property, an over-approximating target admits more behaviors, so proving the property on the target implies it on the source (sound), but a counterexample on the target may be spurious (not complete) — the familiar situation in abstraction-based verification (Cousot & Cousot, 1977; Clarke et al., 2000). An under-approximating target is the mirror image: counterexamples are genuine, but a proof on the target says little about the source. Partial mappings and divergent translations are marked accordingly. The *Neither* and *Not established* columns assert nothing and are therefore compatible with every semantic relationship. The directions above are stated for safety properties; for liveness properties or for property translations that are not monotone, the direction of soundness may reverse, which is why Axis B is always relative to a named property.

Proposition P2 (orthogonality). *(design requirement; not formally proven).* The two axes of the translation classification are orthogonal in the sense that neither is derived from the other and each is recorded separately; the compatibility matrix of Figure 6 constrains which pairs may be recorded

together but does not collapse the axes into one. *Justification sketch:* Axis A concerns the relation between behavior sets, Axis B concerns the relation between provability on T and truth on S for a named property; the same Axis A value can license different Axis B values depending on the property class.

Translation records. Every translation is itself a CSR record with `source`, `target`, `semantic_relationship` (Axis A), `verification_property` (Axis B, relative to a named property), `justification`, `loss_description`, and `assumptions`. A classification is a *claim* and carries its own justification; classifying a translation as Exact does not make it so (threat T9, Section 9).

3.5 Semantic Consistency (Provisional)

Provisional definition. Two CSR records are *semantically consistent* with respect to a property P if there exists a translation between them classified as `Sound` (Axis B) with respect to P, and no `contradicts` relation between them has been recorded as a verification finding.

This definition is provisional and is explicitly an open question (Q2). It is a *relative, record-level* definition: it says when the architecture will treat two records as consistent, not when they actually are. A complete definition would require, first, a formalization of "the meaning" of a specification element in each language, and second, a decision procedure for checking agreement of meanings across languages. Neither exists in this paper. In particular, the provisional definition inherits the weakness that the `Sound` classification is itself asserted: its reliability depends on the justification attached to it and on human review, and it is only as strong as that review.

The relevant open questions are Q1 (under what conditions is a translation sound with respect to a property), Q2 (a mathematically precise definition of semantic consistency), Q3 (the minimal CSR structure preserving semantic identity), Q4 (structure of assumption dependency graphs), Q5 (when an approximation is good enough for a requirement), Q6 (proof portability across tools), and Q7 (whether the three identities can be reconciled mechanically). Appendix B lists all fifty open questions.

Limitations of this section. The definitions in this section are structural. P1 and P2 are design requirements. No translation between TLA+, Why3, Lean, and SMT-LIB is specified or implemented here, and no claim is made that any particular translation is sound. The matrix of Figure 6 is stated for safety properties and relies on a behavior-set view of semantics that not every formalism supports directly.

4. Proof Obligations

4.1 Definition of a Proof Obligation

Definition 4 (proof obligation). A proof obligation is a tuple

```
O = (id, property, assumptions, source_csr, discharged_by, verdict)
```

Field	Type	Description
id	Identifier	Unique, stable identifier of the obligation
property	Formula	The property to be proven
assumptions	Set<AssumptionID>	Assumptions under which the property must hold
source_csr	CSR-ID	The CSR record that generated this obligation
discharged_by	ProofEvidence or none	Evidence that discharges the obligation, if any
verdict	Verdict	Proven / Refuted / Unknown

A note on the verdict range. One source document lists four verdict values including *Escalated*, while the same document later separates verification verdict from workflow disposition. This paper adopts the latter reading (design decision D13): the verdict has three values (*Proven*, *Refuted*, *Unknown*), and escalation to a human is a workflow disposition (Section 4.6), not a verdict.

Properties. An obligation is *typed* (its property is a formula of a specified logic), *assumption-closed* (it depends only on assumptions in its `assumptions` set), *traceable* (to a CSR record and through it to a requirement or an `unformalized` marker), and *dischargeable* (by evidence, or left `Unknown`). **An obligation is a question, not an answer;** it becomes an answer only when discharged by evidence.

4.2 Generation Procedure

Obligations are generated from CSR records according to their `semantic_role`.

CSR semantic_role	Generated obligation
invariant (L1)	Invariance(property, transitions, assumptions)
contract (L2)	PrePost(pre, post, function, assumptions)
constraint (L3)	Satisfiability(constraint, assumptions)
translation	Consistency(source, target, property, assumptions)

Generation rules.

1. **No obligation without a source:** every obligation traces to exactly one CSR record.
2. **No obligation without assumptions:** an obligation may declare an empty assumption set only if tagged axiomatic (approval process deferred to the human review protocol, Section 8).
3. **No obligation without a property:** an obligation without a property is invalid.
4. **No hidden obligations:** every generated obligation is recorded in its source CSR record's relations set.

Illustrative generation. Given a Layer 1 CSR record asserting that a work queue never exceeds a bound, with the single assumption that the bound is positive, the generator produces an Invariance obligation. The fragment below is illustrative; it is not the output of an implementation.

```
csr_id:        csr-L1-001
layer:         L1            language: TLA+
semantic_role: invariant
content:       Always (queue_length <= MAX)
assumptions:   {A1: MAX > 0}

O-001:
  property:      forall s in Reachable : queue_length(s) <= MAX
  assumptions:   {A1}
  source_csr:    csr-L1-001
  discharged_by: none
  verdict:       Unknown
```

4.3 Obligation Dependency Graph (ODG)

Obligations are organized in a directed graph ODG = (Obligations, Edges), where an edge (O_1, O_2) means that O_2 depends on O_1 being `Proven`.

Edge type	Meaning
requires	O_2 requires O_1 to be Proven before it can be discharged
refines	O_2 is a refinement of O_1

Edge type	Meaning
assumes	O_2 assumes O_1 as a hypothesis
contradicts	O_2 contradicts O_1: both cannot be Proven

No cyclic dependencies. The ODG must be a DAG; a cycle indicates a design error in the CSR records and must be resolved before obligation generation (design decision D10). Because `contradicts` is symmetric, it would trivially form a two-cycle; this paper therefore reads the acyclicity requirement as applying to the dependency-bearing edge types (`requires`, `refines`, `assumes`) and treats `contradicts` as a symmetric consistency constraint: if both endpoints of a `contradicts` edge are ever `Proven`, a verification finding is recorded. This is a clarification introduced by the present paper.

Incremental invalidation. If the verdict of O_1 ceases to be `Proven` (for example because its source CSR record changed) and an edge ($O_1 \rightarrow O_2$) exists, then the verdict of O_2 is set to `Unknown`, transitively. Section 5 refines this rule into the full differential model.

Proposition P3 (ODG acyclicity). *(design requirement; not formally proven).* Proof obligations form a directed acyclic graph over the dependency-bearing edge types. *Justification sketch:* acyclicity is a checkable invariant enforced at generation time; any insertion that would create a cycle is rejected and reported as a design error.

4.4 Assumption Tracking

Definition (assumption). An assumption is a named proposition that an obligation depends on but does not itself prove: A = (id, statement, source, verified_by), where `source` is a CSR record, an axiom, or a requirement, and `verified_by` is evidence or none.

Class	Description
Axiomatic	Taken as given; no proof required
Empirical	Verified by experiment or measurement
Derived	Proven from other assumptions or obligations
Environment	Assumed about the environment (for example, that a clock is monotonic)

Change management. (i) Changing an assumption's statement invalidates every obligation that depends on it. (ii) Adding an assumption to an obligation requires recording the addition in the CSR record's `assumptions` set. (iii) Removing an assumption from an obligation is allowed only if no obligation depends on it.

Silent assumption prevention. The system must (1) reject proof evidence whose TCB manifest relies on an assumption the obligation does not declare, (2) log every assumption change with a timestamp and a diff, and (3) prevent assumption changes from being committed without explicit human or tool approval. The source documents phrase the first requirement both as “declares the same assumptions” and as “is a subset of the obligation's assumptions”; this paper adopts the subset reading, i.e., evidence may not rely on any assumption the obligation has not declared, and evidence relying on fewer assumptions is acceptable.

4.5 Discharge Protocol

An obligation can be discharged by any of the following paths.

Path	Tool	Produces
TLA+ model checking	TLC	Bounded exploration; may be Unknown beyond the bound

Path	Tool	Produces
TLA+ proof	TLAPS	Proof term checkable by the TLA+ proof infrastructure
Why3 verification	Why3 + backend provers	Verification conditions discharged by provers
Lean proof	Lean	Proof term checkable by the Lean kernel
SMT solving	Z3 + proof checker	unsat plus a proof artifact, if produced
Human review	Human	Escalated disposition with human-written justification

Discharge requirements. Every discharge must satisfy: (1) *assumption match* as in Section 4.4; (2) *TCB completeness* — the manifest records all components used; (3) an *evidence artifact* (proof term, model, or human justification) is stored; (4) *independent checkability* — where possible, the evidence is checkable by a component not used to produce it. If no path discharges the obligation, the verdict remains `Unknown`, the disposition becomes *Further Analysis* or *Escalated to Human*, and the obligation is recorded in a failure log with its attempt history (design decision D14).

4.5.1 Per-Path TCB Manifests

Each verification path has its **own** TCB manifest, an explicit artifact rather than an implicit assumption (design decision D6, D12). A verification result without a manifest is not admissible evidence.

Path	TCB manifest contents
TLA+ model checking	TLC version, model configuration, state-space bound, exploration strategy
TLA+ proof (TLAPS)	TLAPS version, TLA+ kernel version, backend provers
Why3 verification	Why3 version, transformations, drivers, backend provers
Lean proof	Lean version, kernel version, imported libraries, axioms used
Z3 unsat	Z3 version, SMT-LIB version, proof production enabled, proof artifact hash

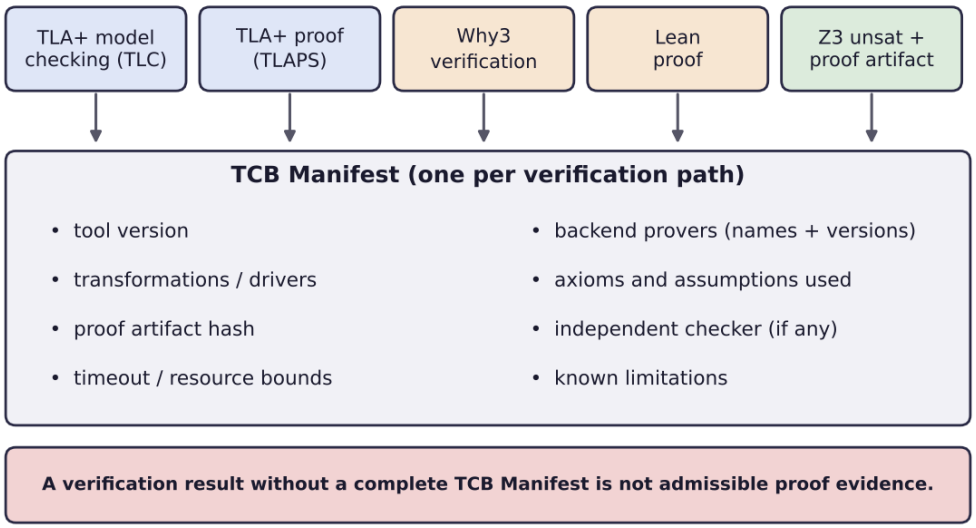


Figure 7 (Fig5_TCB_Manifest.png). Per-path TCB manifests. Each path records tool and prover versions, transformations, axioms and assumptions used, proof artifact hash, independent checker, resource bounds, and known limitations.

4.6 Verdict and Workflow Disposition

Verification verdict and workflow disposition are **orthogonal** (design decision D13): the verdict is a statement about the property (*Proven*, *Refuted*, *Unknown*); the disposition is a statement about what the workflow does next (*Completed*, *Remediation Required*, *Further Analysis*, *Escalated to Human*). The usual mapping is not automatic.

Verdict	Typical disposition
Proven	Completed
Refuted	Remediation Required
Unknown	Further Analysis, or Escalated to Human
Any	Escalated to Human, if an authority policy requires

A *Refuted* verdict may be escalated rather than sent to remediation if the refuting counterexample is disputed. *Unknown* is a legitimate, recorded outcome, not a failure to be hidden.

Limitations of this section. Obligation generation here is specified per role, not algorithmically; the full obligation schema is deferred. Per-path manifests make the trusted base visible but do not shrink it, and the independence requirement in the discharge protocol is stated as “where possible” because not every path has an independent checker. Open questions Q8–Q12 concern the minimal obligation structure, safe re-discharge after assumption change, ODG structure, acceptability of bounded results, and encoding human review as evidence.

5. Incremental Differential Proof Engine

5.1 The Differential Proving Problem

Given a repository with a set of discharged obligations and a new changeset δ (written d where a plain letter is required), the engine must determine: (1) which obligations are **affected** by δ; (2) which affected obligations can be **safely reused**, i.e., whose evidence remains valid; (3) which must be **re-discharged**; and (4) the **minimal set** of re-discharges that restores full evidence coverage.

Full re-proving after every change is prohibitively expensive at agent cadence, so the engine exploits the structure of the changeset DAG and of the dependency graph to avoid re-proving unaffected obligations, reuse evidence whose assumptions are unchanged, and invalidate only what is necessary. The design identifies two opposing failure modes and aims to minimize both rather than eliminate one at the expense of the other.

Failure mode	Consequence
Over-invalidation	Excessive re-proving; performance degrades toward full re-proving
Under-invalidation	Unsound reuse: a stale proof is treated as valid

The asymmetry of costs is worth stating. Over-invalidation wastes effort but preserves soundness; under-invalidation is silent and undermines the purpose of verification. Consequently, when the dependency information is uncertain, the architecture is designed to err toward invalidation and to record the uncertainty, not to reuse optimistically.

5.2 Proof Dependency Graph (PDG)

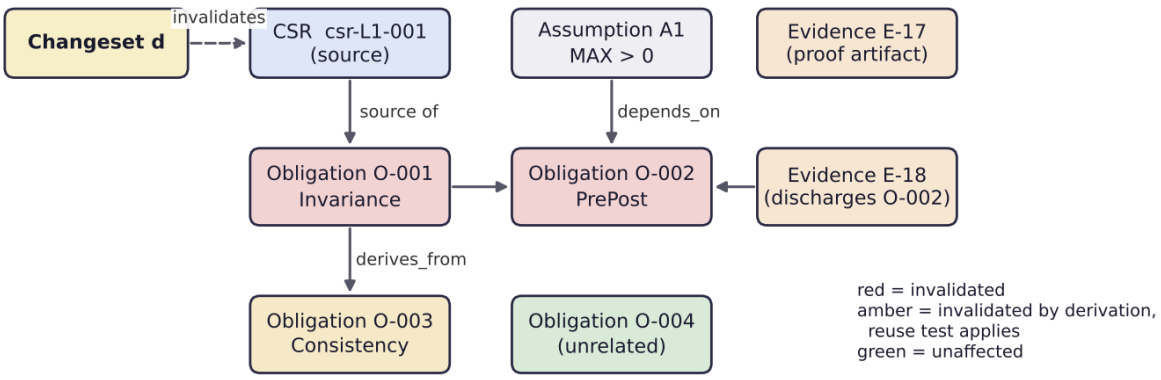
Definition 5 (PDG). The Proof Dependency Graph extends the ODG of Section 4.3 with evidence and assumption nodes: PDG = (Obligations ∪ Evidence ∪ Assumptions, Edges). The source specification names three node types (obligation, evidence, assumption) and also refers to changesets and CSR records through its `invalidates` edges and `source_csr` field; this paper therefore treats **changeset** and **CSR-record** nodes as external anchors of the graph, giving five node types in total.

Node type	Description
Obligation	An obligation to be discharged (Section 4)
Evidence	A proof artifact, model, or human justification
Assumption	A named proposition that obligations depend on
CSR record	Source of obligations (external anchor)
Changeset	A change that may invalidate nodes (external anchor)

Edge type	Direction	Meaning
discharges	Evidence → Obligation	Evidence proves the obligation
depends_on	Obligation → Assumption	Obligation requires the assumption
supports	Evidence → Evidence	One evidence piece supports another
derives_from	Obligation → Obligation	One obligation is derived from another
invalidates	Changeset → Node	A changeset invalidates a node

Invariants.

1. **Acyclicity:** the PDG must be a DAG.
2. **Total discharge:** every Proven obligation has at least one discharges edge from a valid evidence node.
3. **Assumption closure:** every depends_on edge of an obligation points to an assumption declared in its assumptions set.
4. **No orphan evidence:** every evidence node discharges at least one obligation.



Invalidation follows source / depends_on / derives_from edges; unaffected obligations (O-004) retain their evidence.

Figure 8. An illustrative PDG fragment and the effect of a changeset that modifies a CSR record. Red nodes are invalidated; the amber node is invalidated by derivation and is subject to the reuse test of Section 5.4; the unrelated obligation retains its evidence.

Proposition P4 (PDG extends ODG). *(design requirement; not formally proven).* The PDG is obtained from the ODG by adding evidence and assumption nodes and the edge types of the table above, preserving acyclicity. *Justification sketch:* the added edges point from evidence to obligations and from obligations to assumptions or to other obligations; evidence and assumption nodes have no outgoing edges back into obligations other than discharges and supports, so no cycle is introduced by construction, provided the invariants above are enforced.

5.3 Invalidation Rules

A node N is **invalidated** when any of the following triggers holds.

Trigger	Affected nodes
A changeset modifies a CSR record that is the source of N	N and all nodes reachable from N via derives_from
A changeset modifies an assumption that N depends_on	N and all nodes reachable from N via derives_from
A changeset modifies a proof artifact that discharges N	N (the evidence is stale)
A changeset modifies a file within the blast_radius of N	N and all dependents

The **blast radius** of an obligation is the set of files, symbols, and CSR records whose modification could affect its validity. How blast radii are computed and kept sound is an open design matter; the architecture requires only that each obligation declare one, and that guardrails (Section 6.5) use the same notion.

Invalidation algorithm (proposed completion). The source specification gives the algorithm's signature and its first steps; the body below is this paper's proposed completion as a worklist fixpoint.

```
function invalidate(d, PDG):
    seeds       = find_nodes_affected_by(d, PDG)
    invalidated = {}
    worklist    = seeds
    while worklist is not empty:
        n = pop(worklist)
        if n in invalidated: continue
        invalidated.add(n)
        mark_stale(n)
        for m in dependents(n, PDG):      # via derives_from; blast_radius for trigger 4
            push(worklist, m)
    return invalidated
```

Because the PDG is a finite DAG and each node is added to the invalidated set at most once, the loop terminates and visits each node and edge at most once, so its cost is linear in the size of the affected subgraph plus the seed-finding cost. The returned set is the smallest set containing the seeds and closed under the propagation rule; it is minimal *relative to the recorded dependencies*. If a real dependency is not recorded, the algorithm cannot invalidate through it, which is the formal form of the under-invalidation failure mode.

Proposition P5 (invalidation via derives_from). *(design requirement; not formally proven).* Invalidation propagates along derives_from edges (and, for the blast-radius trigger, to all recorded dependents). *Justification sketch:* the invalidated set is the forward closure of the seeds under the propagation rule, computed by the worklist above; soundness with respect to the *actual* semantic dependencies holds only if the recorded graph over-approximates them, which is open question Q13.

5.4 Reuse Conditions

An invalidated obligation O can be **reused** — its evidence remains valid — if and only if all four source conditions hold (a fifth is proposed below):

1 **Assumption preservation:** every assumption in O.assumptions is unchanged.

2 **Evidence compatibility:** the evidence's TCB manifest declares the same tool versions and assumptions as O.

3 **Property stability:** the property O.property has not changed.

4 **Semantic relation preservation:** every refines and approximates relation on the path from O to its source CSR record is unchanged.

If any condition fails, O must be re-discharged. The “if and only if” here is a *design rule* about when the engine will reuse; whether these conditions are also sufficient for the evidence to remain *truly* valid after an arbitrary code change is not established, and is open question Q14.

Proposed fifth condition. Conditions 1–4 concern the specification side only. Taken alone they would permit reusing evidence computed against an implementation that has since changed, which is exactly the case the blast-radius trigger is meant to catch. This paper therefore proposes an additional condition, not present in the source specification: **5. Artifact stability** — the artifacts the evidence was computed against, as recorded by hash in its manifest, are unchanged. The decision procedure below includes it.

```
For each invalidated obligation O:
    if assumptions_unchanged(O) and evidence_compatible(O)
       and property_stable(O) and relations_unchanged(O)
       and artifacts_unchanged(O):    # proposed condition 5
        reuse(O)            # mark as validated
    else:
        re_discharge(O)
```

5.5 Re-Discharge Strategy

Priority. Obligations that cannot be reused are re-discharged in priority order: *high* — obligations on the critical path (for example L1 invariants that gate releases); *medium* — obligations downstream of high-priority ones; *low* — obligations whose source CSR records are new or recently modified. Priority scheduling is also the starvation-prevention mechanism of Section 5.7.

Path selection. For each re-discharged obligation, the engine selects a discharge path.

Condition	Preferred path
Property is decidable in SMT-LIB	Z3 (fast)
Property involves system behavior	TLA+ (TLC or TLAPS)
Property involves functional contracts	Why3
Property requires dependent types	Lean
None of the above	Escalated to Human

The engine estimates proving cost before dispatch and may choose a cheaper but weaker path if the stronger path would exceed budget. Choosing a weaker path must be recorded, because the resulting evidence is weaker and its TCB manifest differs.

5.6 Cost-Aware Proving

Cost component	Estimation method (qualitative)
SMT solving cost	Formula size, theory complexity
Model checking cost	State-space size, bound
Deductive proof cost	Number of subgoals, complexity of lemmas
Human review cost	Manual effort (qualitative)

Proof budget. The engine operates under Budget = (time_budget, cost_budget, human_attention_budget). When the budget is exceeded, the engine marks remaining obligations `Unknown`, escalates to human review, and records the exhaustion in the audit log. **Tiered proving.** The engine starts at Tier 1 and escalates only if lower tiers fail.

Tier	Tool	Cost	Strength
1	Z3	Low	Weak: decidable theories only
2	TLC	Medium	Bounded model checking
3	Why3	Medium-High	Deductive contracts
4	Lean	High	Full dependent types
5	Human	Very high	Any property

The cost and strength entries are qualitative design statements, not measurements. The tier ordering encodes a policy — try the cheapest applicable evidence first — rather than a claim that tiers form a strict hierarchy of logical strength; the layers of Section 3 are functional categories, and the tiers here are a cost ordering of *tools*. When to escalate rather than accept `Unknown` is open question Q16, and cost estimation for complex obligations is Q15.

5.7 Concurrency and Audit

Decentralized, stateless workers. Proof workers hold no state about the PDG; the PDG is shared through the CSR store; concurrent re-discharges of the same obligation are idempotent. Because workers take no locks on the PDG and the graph is a DAG, deadlock of the lock-cycle kind cannot arise from graph structure; this does not eliminate resource exhaustion or starvation, which are handled by budgets and priority scheduling.

Conflicting evidence. If two workers produce different evidence for the same obligation: both are recorded; the evidence with the stronger TCB is preferred; if TCBs are incomparable, both are retained as alternative evidence; and the choice of authoritative evidence is recorded in the audit log. What “stronger TCB” means formally is open (Q17).

Proof Evidence Record field	Content
obligation_id	The obligation discharged
evidence_type	proof_term / model / human_justification
evidence_artifact	Path or hash of the evidence
tcb_manifest	The TCB manifest used
timestamp, worker_id	When and by which worker the evidence was produced
validation_status	valid / stale / disputed

Audit and re-validation. Every PDG mutation (invalidation, reuse, re-discharge) is recorded in an append-only audit log checked into the repository with the CSR records. Periodically, a re-validation pass checks that every `Proven` obligation has a valid evidence record, that every manifest is still accurate, and that no assumption was silently changed. Re-validation is periodic, not continuous (design decision D21); whether it can be made incremental is open question Q18.

Limitations of this section. Section 5 specifies an engine model, not an engine. No cost model, scheduling policy, or reuse rule has been evaluated; the sufficiency of the reuse conditions (Q14) and the soundness of recorded dependencies (Q13) are open. The invalidation algorithm above is a proposed completion of a partially specified procedure.

6. Counterexample-Guided Remediation

6.1 Definition of a Counterexample

Definition 6 (counterexample). A counterexample is a tuple

```
C = (id, obligation, witness, refutation_evidence, type, provenance)
```

Field	Type	Description
id	Identifier	Unique, stable identifier
obligation	ObligationID	The obligation that was refuted
witness	Model or Trace	The concrete witness violating the property
refutation_evidence	ProofEvidence	Evidence that the witness is valid
type	CounterexampleType	Classification (below)
provenance	Provenance	Tool, timestamp, worker

Type	Description	Remediation direction
Code defect	The implementation violates the specification	Patch the code
Specification error	The specification is too strong or wrong	Revise the specification
Assumption violation	A stated assumption is false in the actual environment	Revise the assumption or the environment
Model error	The proof model is inaccurate	Improve the model
Flaky refutation	The refutation is not reproducible (flaky test or tool bug)	Re-run or mitigate flakiness
Unknown	Type cannot be determined from available evidence	Escalate to human

Key principle. The counterexample type is **not** determined by the tool that produced it. It is determined by the diagnosis protocol of Section 6.3 (design decision D22). A model checker that outputs a trace has found a trace; whether that trace indicates a bug, a bad specification, or a bad model is a separate judgment.

6.2 The Remediation Cycle

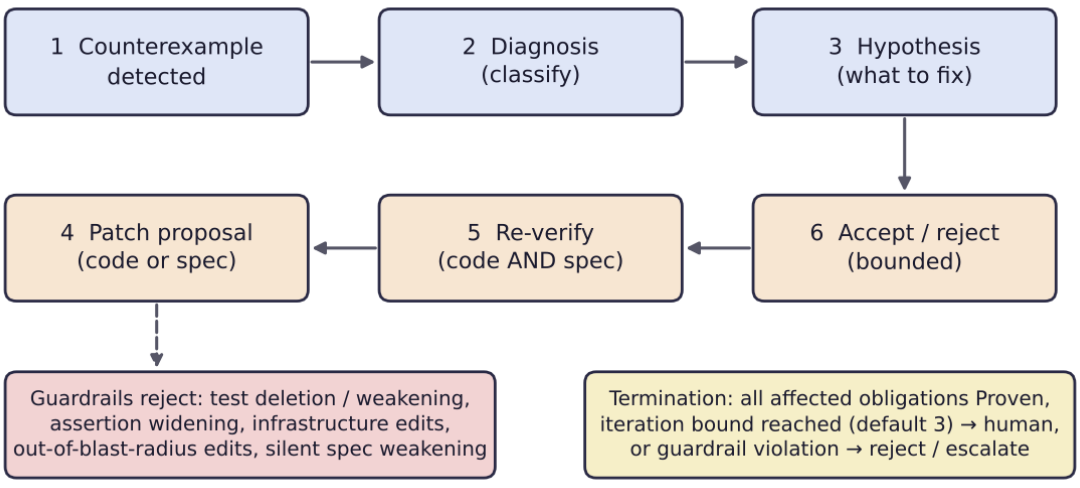


Figure 9. The six-stage remediation cycle with its guardrails and termination conditions.

The cycle has six stages: (1) counterexample detected; (2) diagnosis; (3) hypothesis about what to fix; (4) patch proposal (code or specification); (5) re-verification of code *and* specification; (6) accept or reject, within a bound.

Bounded iterations. The cycle is bounded by a configurable maximum number of iterations (default: three). If no resolution is found within the bound, the counterexample is marked *Escalated to Human*, the full history of attempts is recorded, and silent acceptance of the counterexample is not allowed (design decision D23).

No silent specification modification. A patch that modifies a specification must be explicitly tagged as a specification change and pass through the same review as any other (Section 8.4). A patch that modifies *both* code and specification is rejected unless the reviewer explicitly approves the specification change (D24).

Proposition P6 (bounded remediation). *(design requirement; not formally proven).* The remediation cycle for a given counterexample performs at most b iterations, where b is the configured bound, after which the counterexample is escalated. *Justification sketch:* an iteration counter is incremented at each pass through stage 4; when it reaches b the cycle exits to escalation. Termination of each iteration additionally requires that every stage terminates, which the architecture obtains from per-tool timeouts and the proof budget. Whether any particular bound preserves soundness is open (Q22).

6.3 Diagnosis Protocol

Diagnosis is a decision procedure over the counterexample. The source specification gives the following tree.

```
Given counterexample C:
  if C.witness is not reproducible:
      return "Flaky refutation"
  if C refutes a stated assumption:
      return "Assumption violation"
  if C.witness is a valid model of the actual implementation:
      if C refutes the specification:
          return "Specification error"
      else:
          return "Code defect"
  else:
      return "Model error"
```

Reproducibility check. A counterexample is reproducible if re-running the same discharge path on the same input yields the same witness. Flaky refutations are excluded from remediation (flakiness is treated in Section 9, T10; cf. Luo et al., 2014). **Assumption check.** An assumption violation is detected when the witness falsifies a stated assumption in the obligation's `assumptions` set; this is a diagnosis, not a defect. **Model validation.** A model error occurs when the witness cannot be realized in the actual implementation, indicating that the proof model over-approximates the implementation (Axis A of Section 3.4).

Observations on the tree. The tree is order-dependent: a witness that is both irreproducible and assumption-violating is classified as flaky. The branch “C refutes the specification” is not defined operationally in the source; read literally it asks whether the witness shows the specification, rather than the implementation, to be at fault, which presupposes a notion of intent that the CSR traceability chain only partly supplies. These are limitations of the protocol, recorded as open questions Q19 (minimal diagnosis protocol) and Q20 (when a counterexample is reproducible enough to act on).

DOI: https://doi.org/10.5281/zenodo.24013124

6.4 Patch Proposal

Code patch (for a code defect): modifies the code to satisfy the specification; does not weaken the specification; does not modify tests, test infrastructure, or assertions; stays within the obligation's blast radius. **Specification patch** (for a specification error): revises the specification to reflect actual intent with justification; is tagged as a specification change; is submitted for human review before acceptance. **Assumption patch** (for an assumption violation): either revise the assumption to reflect the actual environment, revise the implementation to satisfy the original assumption, or mark the obligation *Escalated* if neither is acceptable.

6.5 Guardrails

Every patch proposal passes through guardrails. A patch that fails any guardrail is rejected and the cycle returns to diagnosis with a new hypothesis.

Guardrail	Rejection condition
Test integrity	Patch deletes, disables, or modifies a failing test
Assertion integrity	Patch widens an assertion's tolerance
Infrastructure integrity	Patch modifies test infrastructure
Blast radius	Patch modifies files outside the declared blast radius
Specification integrity	Patch weakens the specification without explicit approval

Why guardrails are central. A remediation agent is optimized, directly or indirectly, to make a failing check pass. The cheapest ways to do so are often illegitimate: delete or relax the test; weaken the invariant; add an assumption that is false in practice so that the property holds *vacuously*; or drop the obligation. The guardrails map onto these moves. The first three block edits to the checking apparatus; the blast-radius guardrail confines the patch; and the specification-integrity guardrail, together with the assumption-tracking rules of Section 4.4, makes assumption inflation and invariant weakening visible and subject to review. These guardrails are a **superset** of those in AetherFlow's remediation policy (Baladi, 2026c, Section 4.4), adding specification-integrity checks, assumption-change tracking, and counterexample-specific validation. How to formalize guardrails so that gaming is provably excluded, rather than merely discouraged, is open question Q23.

6.6 Re-Verification

Every accepted patch triggers: **code re-verification** (the original obligation is re-discharged against the patched code); **specification re-verification** (if the patch modified the specification, it is re-validated against its source CSR record); and **assumption re-verification** (if assumptions changed, all dependent obligations are re-checked).

```
For each affected obligation O:
    if patch modifies O.source_csr:
        re_discharge(O) with new CSR
    else:
        re_discharge(O) with patched code
    if re_discharge(O).verdict == Refuted:
        return to Remediation Cycle
```

The cycle terminates when all affected obligations are `Proven`, when the iteration bound is reached (escalate), or when a guardrail violation is detected (reject the patch). Whether the cycle is confluent — whether the outcome is independent of the order in which counterexamples are remediated — is open question Q24.

6.7 Escalation to Human

Escalation trigger	Reason
Iteration bound reached	The engine cannot resolve the counterexample
Guardrail violation detected	The patch would compromise verification integrity
Specification change proposed	Specification changes require human approval
Conflicting evidence	Multiple counterexamples contradict each other
Unknown counterexample type	Diagnosis is inconclusive

Escalation record. Every escalation produces a record with: `counterexample_id`, `diagnosis_history`, `patch_attempts` (accepted and rejected), `reason`, `timestamp`, and `human_decision` (to be filled by the reviewer). The reviewer may *accept* the counterexample as a real defect and propose a fix, *reject* it as spurious (with justification), *revise* the specification (with new traceability), or *defer* it to future work (with justification).

Limitations of this section. The remediation protocol is specified, not implemented, and no remediation success rate is claimed. The diagnosis tree is order-dependent and partly under-specified (Q19, Q20); guardrails discourage but do not provably prevent specification gaming (Q23); the default iteration bound of three is a design default, not a tuned value (Q22); and confluence is open (Q24). Related work on automated repair (Le Goues et al., 2012) and on agent evaluation (Jimenez et al., 2024) informs the problem but does not validate this protocol.

7. Integration with AetherFlow

7.1 Design Principles

Principle	Rationale
Protocol-based, not code-based. The systems communicate by messages, not shared libraries.	Each system can evolve independently; a change in AetherProof's internal algorithms does not break AetherFlow so long as the protocol is respected.
Asynchronous, not blocking. Neither system blocks waiting for the other.	Proving an obligation may take minutes; AetherFlow admission must not block (cf. AetherFlow Proposition 1).
Versioned, not frozen. The protocol carries a version; each system declares supported versions.	The two systems can be upgraded independently.
Failure-tolerant, not failure-free. The protocol specifies failure semantics for each message type.	Systems fail; failure must not corrupt state.

7.2 Message Types

Every message has the form Envelope = (version, message_id, timestamp, source, destination, payload). The catalogue comprises ten messages.

#	Message	Direction	Purpose
M1	changeset_admitted	AetherFlow → AetherProof	Notify of a new changeset
M2	verification_request	AetherFlow → AetherProof	Request proof of a property
M3	verification_result	AetherProof → AetherFlow	Return verdict and evidence
M4	counterexample_found	AetherProof → AetherFlow	Notify of a refuted property
M5	remediation_patch	AetherProof → AetherFlow	Submit a remediation patch
M6	escalation	AetherProof → AetherFlow	Escalate a case to human review

#	Message	Direction	Purpose
M7	evidence_query	AetherFlow → AetherProof	Query for existing evidence
M8	evidence_response	AetherProof → AetherFlow	Return existing evidence
M9	health_check	Both	Verify liveness
M10	version_negotiation	Both	Negotiate protocol version

Illustrative payloads follow. They are examples of structure, with placeholders in place of any real tool version or identifier; the full message schema is deferred.

```
M1 changeset_admitted
{ "changeset_id": "cs-<id>", "parent_ids": ["cs-<id>"],
  "delta_hash": "<hash>", "timestamp": "<time>" }

M3 verification_result
{ "obligation_id": "O-001", "verdict": "Proven",
  "evidence_type": "proof_term", "evidence_artifact": "<path-or-hash>",
  "tcb_manifest": { "tool": "<tool>", "version": "<x.y.z>",
                    "backend_provers": ["<prover>"], "timestamp": "<time>" } }

M4 counterexample_found
{ "obligation_id": "O-002",
  "counterexample": { "witness": "queue_length = MAX + 1",
                      "refutation_evidence": "<path>", "type": "code_defect" } }
```

Relation to AetherFlow's verification operator. In AetherFlow, the status of a changeset is computed from its delta, its parents' statuses, and CI results. This paper proposes that AetherProof's M3 and M4 messages be consumed as a particular kind of CI evidence: a *Refuted* obligation attributable to the changeset's delta is a failing check attributable to that delta, and a *Proven* obligation is a passing check. A *Unknown* obligation must **not** be treated as passing: if an obligation is declared gating, AetherFlow should withhold advancing the verified head on *Unknown* (or treat it according to a stated policy), rather than silently promoting it. This mapping is a design proposal; it has not been agreed with, or tested against, any AetherFlow implementation.

7.3 Version Negotiation

The protocol version follows semantic versioning (MAJOR.MINOR.PATCH): a MAJOR change is a breaking change in message formats; MINOR adds message types backward-compatibly; PATCH is a backward-compatible fix. On startup both systems declare supported versions, the highest common MAJOR.MINOR is selected, and if none exists the systems refuse to communicate. Systems ignore messages of unsupported MINOR types; MAJOR differences require negotiation or fallback.

7.4 Failure Semantics

Failure	Detection	Response
AetherProof unavailable	Health-check timeout	AetherFlow continues without proof (degraded)
AetherFlow unavailable	Health-check timeout	AetherProof queues messages (bounded)
Message lost	No acknowledgment	Retry with exponential backoff
Message corrupted	Hash mismatch	Reject and log
Version mismatch	Negotiation failure	Refuse communication

Degraded mode. If AetherProof is unavailable, AetherFlow continues admission and verification — its core function — while proof obligations are queued in AetherProof's inbox and processed on return; messages are durably stored so no data is lost. A consequence worth stating: in degraded mode, the

verified head advances on CI evidence alone, without proof evidence, and the evidence record should say so. **Bounded queues.** Both systems maintain bounded queues; when a queue is full, new messages are rejected, the sender is notified (backpressure), and retries later. How backpressure should be communicated is open (Q32).

7.5 Security and Trust

Authentication: messages are signed by the sender's key and verified before processing. **Authorization:** AetherProof may only *submit* remediation patches — it cannot modify code directly; AetherFlow may only *request* verification — it cannot modify CSR records; specification changes require human review. **Audit:** every message is logged with sender, timestamp, and hash; the audit log is append-only and tamper-evident and is shared by both systems.

Boundary	Trust level
AetherFlow ↔ AetherProof	Medium (protocol-based, authenticated)
AetherProof ↔ external tools	Low (tools are treated as untrusted)
AetherProof ↔ human	High (human review is authoritative)
AetherProof ↔ CSR store	High (the CSR store is a trusted component)

7.6 Integration Examples

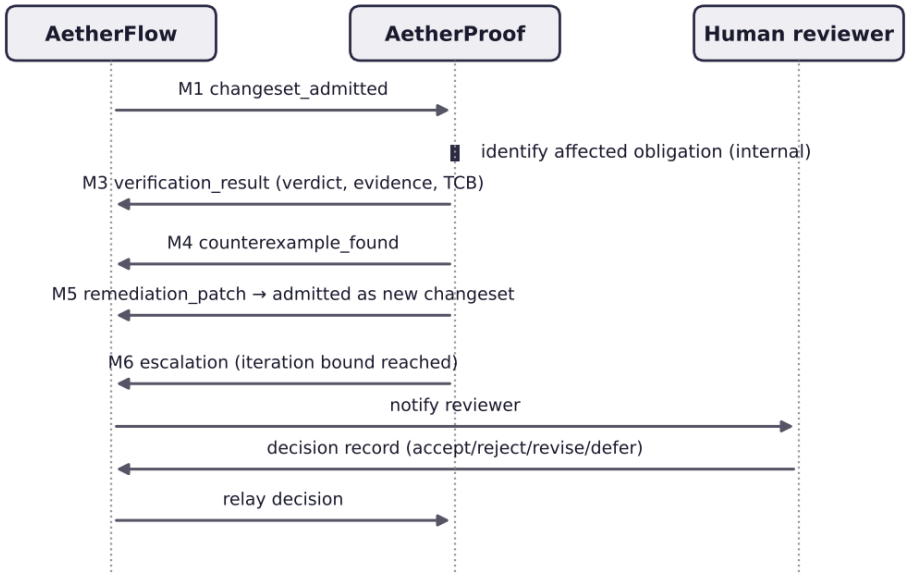


Figure 10. Message flow for the three integration scenarios: verification (M1, M3), refutation and remediation (M4, M5), and escalation (M6) with a human decision relayed back.

Example 1 — successful verification. AetherFlow admits a changeset and sends M1. AetherProof identifies an affected obligation, discharges it (for example via TLA+), and returns M3 with evidence and manifest; AetherFlow records the verified status. **Example 2 — refuted obligation.** AetherProof discharges an affected obligation and obtains *Refuted*; it sends M4, diagnoses the counterexample (say, a code defect), sends M5 with a patch that passed the guardrails; AetherFlow admits the patch as a new changeset; AetherProof re-verifies the obligation on the patched code. **Example 3 — escalation.** AetherProof cannot resolve a counterexample within three iterations and sends M6; AetherFlow notifies a human reviewer, who examines the counterexample and patch attempts and records a decision (accept, reject, revise, defer) that AetherFlow relays back.

Limitations of this section. The protocol is a design. No message schema is final, no latency or throughput figure is claimed, and no integration with any running AetherFlow system has been performed. Open questions Q31–Q36 concern the minimum protocol version, backpressure, message size, audit-log merging after disconnection, multiple AetherProof instances, and latency SLAs.

8. Human Review Protocol

Human review is a first-class component of AetherProof, not an exception path (design decision D32). The architecture does not seek to automate human judgment, nor to reduce review to a rubber stamp; it seeks to say *when* judgment is required, *what evidence* the reviewer must see, and *how the decision is recorded* so that it can be audited.

8.1 Five Roles

Role	Authority	Responsibility
Specification Author	Write, revise specifications	Ensure specifications reflect intent
Verification Engineer	Discharge obligations, propose patches	Ensure proofs are sound
Reviewer	Approve specification changes, accept remediations	Ensure decisions are justified
Maintainer	Merge approved changes	Ensure the repository remains coherent
Auditor	Inspect decisions, question evidence	Ensure accountability

Authority matrix. “yes” = authority; “escalate” = authority exercised only through escalation (a stricter, double-signed path); “no” = no authority.

Decision	Author	Verif. Eng.	Reviewer	Maintainer	Auditor
Write spec	yes	no	no	no	no
Revise spec (minor)	yes	no	yes	no	no
Revise spec (major)	yes	no	escalate	yes	no
Discharge obligation	no	yes	no	no	no
Accept remediation (code)	no	yes	yes	no	no
Accept remediation (spec)	no	no	escalate	yes	no
Escalate to human	no	yes	yes	no	yes
Merge changes	no	no	no	yes	no
Question evidence	no	no	no	no	yes

8.2 Separation of Duties

1. No single person may hold both the **Specification Author** and the **Reviewer** role for the same specification.
2. No single person may **discharge an obligation** and **approve its own evidence**.
3. The **Maintainer** must be distinct from the Verification Engineer who produced the patch.

The rationale is the conventional one: separation of duties prevents self-approval, which would undermine the integrity of the verification chain. The minimal role set that preserves separation is open (Q37), as are the detection and handling of reviewer conflicts of interest (Q38).

Proposition P8 (separation of duties). *(design requirement; not formally proven).* The architecture requires that no single person approves a decision they originated: the author/reviewer, discharger/approver, and maintainer/patch-producer pairs are each disjoint. *Justification sketch:* the three rules above are checkable against role assignments recorded in decision records; the proposition states a requirement on the process, not a property that holds automatically.

8.3 Escalation Triggers

Automatic escalation. The following triggers automatically require human review.

Trigger	Reason	Reviewer role
Specification change	Any change to a CSR record	Specification Author + Reviewer
Assumption change	Any modification to an assumption	Reviewer
Counterexample unresolved	Iteration bound reached (Section 6.2)	Reviewer
Guardrail violation	Patch would weaken verification (Section 6.5)	Reviewer + Maintainer
Conflicting evidence	Two pieces of evidence disagree	Reviewer + Auditor
Unknown counterexample type	Diagnosis inconclusive	Reviewer
Axiomatic specification	Specification marked axiomatic	Reviewer + Auditor
Trust boundary change	A new tool enters a verification path	Reviewer + Auditor

Proposition P7 (specification changes require human review). *(design requirement; not formally proven).* Any change to a CSR record, including one proposed by a remediation engine, requires human review before it takes effect. *Justification sketch:* the specification-change trigger above is unconditional, and the remediation protocol rejects patches that modify a specification without explicit reviewer approval (Section 6.2).

Voluntary escalation. Any role may escalate a case it believes requires judgment beyond the automated system; voluntary escalation must be recorded with a reason and timestamp. **Non-escalation.** The following do not require review: routine obligation discharge (automated proof is sufficient), minor code patches with passing guardrails (automated verification suffices), evidence queries (read-only), and health checks (infrastructure).

8.4 Review Workflows

Specification change review. The author proposes a change. The reviewer receives the proposal with evidence (CSR record, affected obligations, proofs) and examines: is the change traceable to a requirement; does it preserve semantic consistency (Section 3.5); are assumptions explicit; are affected obligations re-discharged. The reviewer approves (the maintainer merges), rejects (back to the author), or requests more evidence (back to the verification engineer).

Remediation acceptance review. The verification engineer proposes a patch. The reviewer receives patch, counterexample, and evidence and examines: does the patch address the diagnosed type; do guardrails pass; is it within the blast radius; does it weaken any specification. The reviewer approves (the changeset is emitted to AetherFlow), rejects (back to the verification engineer), or escalates.

Assumption revision review. The author proposes a revision. The reviewer receives it with the list of dependent obligations and examines: is the new assumption justified (empirical, derived, or axiomatic); are all dependent obligations re-discharged; does the revision change the trust boundary. The reviewer approves (dependents are re-discharged), rejects, or escalates to a trust-boundary review.

DOI: https://doi.org/10.5281/zenodo.24013124

Escalation review. The reviewer receives the escalation record (counterexample, attempts, reason) and accepts the counterexample and proposes a fix, rejects it with justification, revises the specification (entering the specification-change workflow), or defers it as future work, recording the decision with rationale.

8.5 Evidence Requirements per Review Type

Review type	Required evidence (source)
Specification change	CSR record old and new (CSR store); affected obligations (PDG); re-discharged obligations (proof engine); traceability chain (CSR store); assumption list (CSR store)
Remediation acceptance	Counterexample record (analyzer); diagnosis (diagnosis protocol); patch diff (remediation engine); guardrail results; re-verification evidence (proof engine)

Evidence sufficiency. A reviewer may not approve a decision without the required evidence; may reject a decision on evidence insufficiency alone; and must record why evidence is insufficient.

8.6 Decision Recording

Every human review decision produces a signed Decision Record. Decision Records are append-only; any modification is appended as a new record and never overwrites an old one; the log is checked into the repository alongside the CSR store.

```
{ "decision_id": "DR-<id>", "reviewer": "<reviewer-id>",
  "decision_type": "specification_change | remediation | assumption_revision | escalation",
  "subject": "csr-L1-001", "decision": "approved | rejected | escalated",
  "rationale": "<text>", "evidence_reviewed": ["csr-L1-001-v1", "csr-L1-001-v2", "O-001"],
  "timestamp": "<time>", "signature": "<signature>" }
```

Decisions are auditable by the Auditor role, by AetherFlow's audit log, and, if the repository is public, by the public. Whether human review can be made auditable without slowing the process is open (Q40).

8.7 Human Review as Discharge Evidence

Some obligations cannot be discharged by any automated path. In such cases human review may serve as discharge evidence, but only when: (1) no automated path can discharge the obligation; (2) the human has the Reviewer role; and (3) the reviewer explicitly signs the discharge. The evidence is recorded with type `human_justification`, the reviewer, the justification, references, timestamp, and signature.

Limitations. Human review evidence does not prove a property; it *accepts* a property as proven under human judgment. It is not portable across systems, unlike a Lean proof term. It must be re-validated if the specification changes.

Proposition P9 (human discharge only when automation cannot). *(design requirement; not formally proven).* Human review may discharge an obligation only when no automated discharge path can. *Note on formulation:* an earlier statement of this proposition reads “only if undecidable”; the operative rule in the protocol is the weaker, operational one used here, because an obligation may be decidable in principle yet beyond the available tools. By Rice's theorem (Rice, 1953), non-trivial semantic properties of programs are undecidable in general, so undecidability alone is not a usable trigger. *Justification sketch:* the three conditions above are checkable at discharge time.

8.8 Trust Hierarchy

Level	Trust	Examples
Tool-verified	High	Lean kernel; Z3 unsat with proof artifact
Human-reviewed	Medium-High	Reviewer-approved specification change
Human-justified	Medium	Human review as discharge evidence
Empirical	Medium	Measurements, experiments
Axiomatic	Variable	Depends on justification
Unknown	Low	Unresolved obligations

Trust is explicit, not implicit: it does not propagate automatically between levels.

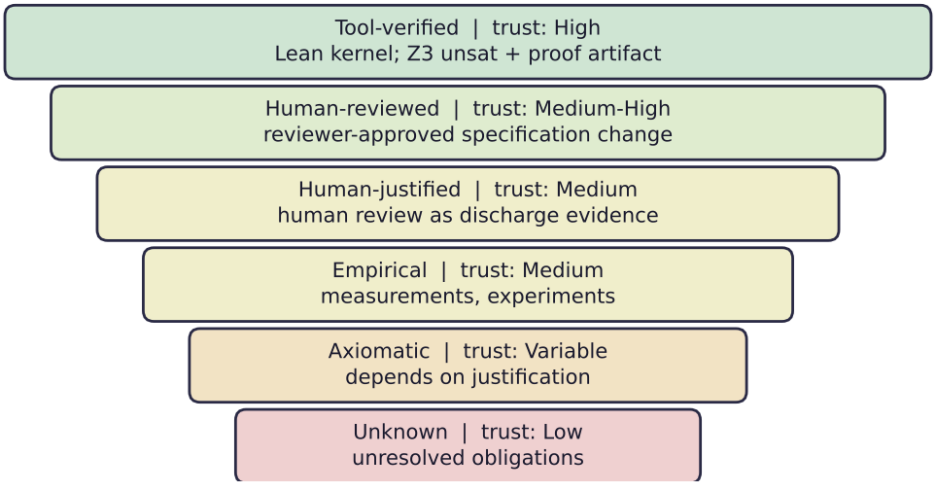


Figure 11. The trust hierarchy. Levels are a classification of evidence, not a ranking of correctness, and trust does not propagate automatically between levels.

Trust propagation. Trust does not propagate automatically: a tool-verified proof is not trusted if its manifest is incomplete; a human-reviewed change is not trusted if the reviewer has a conflict of interest; an axiomatic assumption is not trusted if it is unjustified.

Proposition P10 (trust is explicit). *(design requirement; not formally proven).* Trust in any piece of evidence is a recorded attribute derived from its TCB manifest, reviewer, and justification, and is never inferred from the evidence's position in a pipeline. *Justification sketch:* every evidence record carries a manifest or a signed justification (Sections 4.5, 8.7); trust level is computed from these, not inherited.

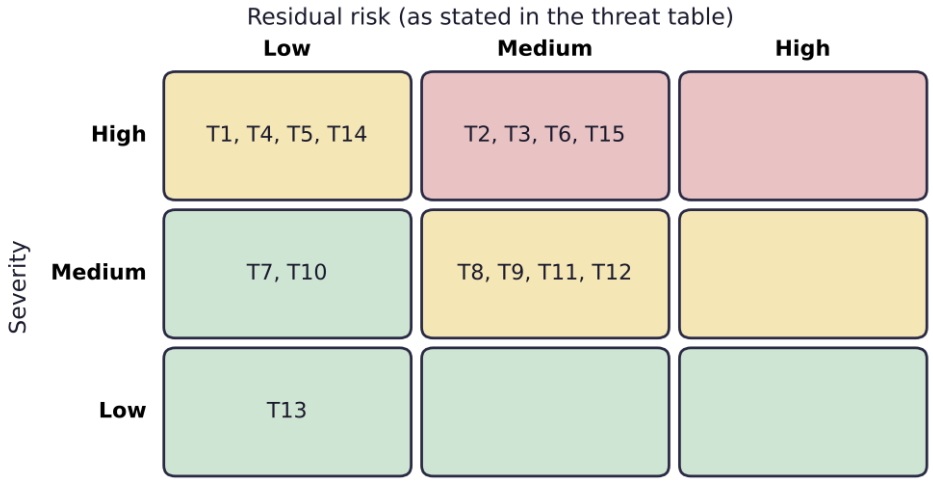
Limitations of this section. The protocol specifies roles and records, not an identity system, conflict-of-interest detection, or cognitive-load management; these are deferred (DD34–DD38). The cognitive burden of reviewing specification changes at scale is unknown (Q39), and human review is fallible: nothing in this section guarantees reviewer correctness. Open questions Q37–Q42 concern roles, conflicts, burden, auditability, re-validation of human evidence, and ML-assisted triage.

9. Threats and Limitations

9.1 Threat Taxonomy

Fifteen threats (T1–T15) are catalogued across seven categories, each with a severity, mitigation, and residual risk. The full table is in Appendix D; Figure 12 places each threat by severity and residual risk. The purpose is not to undermine the architecture but to prevent overclaiming: a mitigation that exists on paper reduces a risk only if it is enforced, and residual risk is acknowledged explicitly (design decisions D48–D52).

Category	Description
Soundness threats	The system may accept an invalid proof
Completeness threats	The system may reject a valid proof
Trust threats	The system may overstate the confidence of a result
Specification threats	The specification may not reflect intent
Process threats	The process may be bypassed or compromised
Toolchain threats	The tools may be incorrect or compromised
Human threats	Human reviewers may fail or be compromised



Qualitative classification from the design documents; T15's residual risk is Medium in the table and High if human review is superficial.

Figure 12. Qualitative placement of the fifteen threats by severity and stated residual risk. This is a classification from the design documents, not a measured risk assessment.

9.2 Soundness Threats

T1 — Unsound proof artifact. A proof artifact accepted as evidence is not actually valid, through a tool bug (for example an `unsat` without a checkable proof), a proof-checker bug, or a toolchain mismatch (a proof produced under one version is checked under another). Mitigations: independent checking, the TCB manifest, and version pinning. Residual risk: low if the checker is trusted and the manifest complete; it is the checker, ultimately, that is trusted (Thompson, 1984). **T10 — Counterexample flakiness.** A counterexample is not reproducible because of flaky tests (Luo et al., 2014), tool nondeterminism such as timeouts, or environmental differences. Mitigations: the reproducibility check and re-running failing checks against the parent state. Residual risk: low if the check is reliable.

9.3 Completeness Threats

T8 — Incomplete completeness proof. The system is taken to be complete when it is not, producing false confidence from the absence of refutation. The system proves only some properties, and an `Unknown` verdict may be ignored. Mitigation: explicit `Unknown` verdict and human review for `Unknown` obligations. Residual risk: medium, and higher if `Unknown` is treated as `Proven`. **T11 — Model over-approximation.** The proof model admits behaviors the implementation does not, yielding false `Refuted` verdicts, whether from L1 models over-approximating the system or from over-approximating translations between layers. Mitigations: model validation and explicit translation classification. Residual risk: medium if models are not validated.

9.4 Trust Threats

T2 — Incomplete TCB manifest. A manifest omits a tool version, assumption, or transformation. Mitigations: per-path manifests, mandatory field completion, and audit of manifests. Residual risk: medium if the audit is not performed. **T4 — Silent assumption change.** An assumption changes without invalidating its dependents, through a tool bug, an unrecorded human edit, or process bypass. Mitigations: assumption tracking, invalidation rules, and signed decision records. Residual risk: low if tracking is enforced. **T9 — Semantic inconsistency between layers.** Two layers have inconsistent meanings that go undetected, because a translation is unsound, or is misclassified as Exact when lossy, or consistency is simply never verified. Mitigations: explicit and two-axis classification; the open question Q2. Residual risk: medium until Q2 is resolved.

9.5 Specification Threats

T3 — Specification does not reflect intent. The specification is formally correct but is not what was meant; for instance it was synthesized from code and is bug-compatible, written without reference to requirements, or not reviewed. Mitigations: the traceability chain, human review, and the `unformalized` marker. Residual risk: medium if review is insufficient. A proof of a wrong specification is a correct proof of the wrong thing, and no part of AetherProof can detect this without human judgment. **T15 — LLM-generated specification error.** A specification generated by a language model contains errors that review does not catch, through hallucination, bias, or overgeneralization. Mitigations: human review and traceability; using multiple models is mentioned but not specified. Residual risk: stated as medium in the table and high if review is superficial.

9.6 Process Threats

T5 — Guardrail bypass. A patch bypasses the guardrails because they are unenforced, weakened by a human, or circumvented by a tool. Mitigations: mandatory guardrails, separation of duties, and audit. Residual risk: low if enforced. **T13 — Denial of service on the proof engine.** An adversary or a buggy synthesis engine floods the engine with obligations, or a tool loops. Mitigations: bounded queues and the proof budget; rate limiting is named but not specified. Residual risk: low if queues are bounded. **T14 — Audit-log tampering.** The log is modified to hide a decision, by a malicious maintainer, an append-only bug, or storage corruption. Mitigations: append-only and signed logs; hash chaining is named but not specified. Residual risk: low if hash chaining is implemented.

9.7 Toolchain Threats

T6 — Toolchain compromise. A tool in a verification path is compromised, through a supply-chain attack, a malicious update, or misconfiguration, and produces invalid proofs or invalid manifests. Mitigations: manifests, version pinning, and independent checking; signed tools are named but not specified. Residual risk: medium if tool signing is not implemented.

9.8 Human Threats

T7 — Reviewer conflict of interest. A reviewer with a conflict (for example, the specification's author) approves a change that should be rejected. Mitigation: separation of duties; conflict-of-interest disclosure is named but not specified. Residual risk: low if separation is enforced. **T12 — Hypothesis-testing bias.** Reviewers or the system favor a hypothesis (confirmation, anchoring, groupthink) and misdiagnose. Mitigations: the structured diagnosis protocol; blind review and multiple reviewers are named but not specified. Residual risk: medium.

Mitigations named but not specified. The threat analysis refers to several mitigations that the current architecture does not define: use of multiple language models (T15), rate limiting (T13), hash-chained audit logs (T14), signed tools (T6), conflict-of-interest disclosure (T7), and blind or multi-person review (T12). The residual-risk ratings that depend on them should be read as conditional on their future specification.

9.9 Threats to Validity of a Future Empirical Evaluation

Validity type	Threats
Internal	Selection bias (non-representative benchmark repositories); measurement bias (mis-measured metrics such as proof time); confounding factors (uncontrolled tool-performance factors); experimenter bias
External	Population validity (benchmarks do not represent all codebases); ecological validity (experimental setting differs from practice); temporal validity (tools and repositories evolve)
Construct	Metric validity (e.g., “proof coverage” may not measure what it claims); definition validity (e.g., “semantic consistency” may not capture the intended concept)
Conclusion	Statistical validity (insufficient sample size); causal validity (correlation mistaken for causation)

9.10 Limitations

Kind	Limitation
Architectural	No implementation benchmarked; no real-world deployment; no empirical validation; protocol-only integration, so cross-system performance is unknown
Design	No universal specification language; no automated specification synthesis from intent; no automated proof of specification correctness; no fully automated remediation
Scope	Formal properties only (empirical properties such as performance are not covered); code only (data, configuration, infrastructure not covered); synchronous systems, with distributed and asynchronous systems only partially covered

Limitations of this section. The threat catalogue is a structured judgment of the author, not the output of a systematic threat-modelling exercise, and may be incomplete. Severity and residual-risk ratings are qualitative and have not been validated. Open questions Q43–Q50 concern tool signing, conflict-of-interest automation, blind review, toolchain evolution, prioritization of Unknown obligations, human-review latency, and extension to distributed systems and empirical properties.

10. System Architecture

This section integrates the specification, obligation, proving, and remediation components of Sections 3–6 into one architecture, states its interfaces, and records what it does not attempt.

10.1 Position in the Aether Family

AetherProof is the verification layer, above the integration layer (AetherFlow) and the storage layer (AetherGit) (Figure 2). The layers answer different questions: AetherGit — how are changes stored and transferred? AetherFlow — how are changes admitted, verified, and remediated? AetherProof — what conditions make behavior correct, and how are they proven? Each layer is methodologically independent, and integration is by protocol (Section 7); AetherProof reads the changeset DAG through a read-only interface and writes only through AetherFlow's admission protocol.

10.2 Components

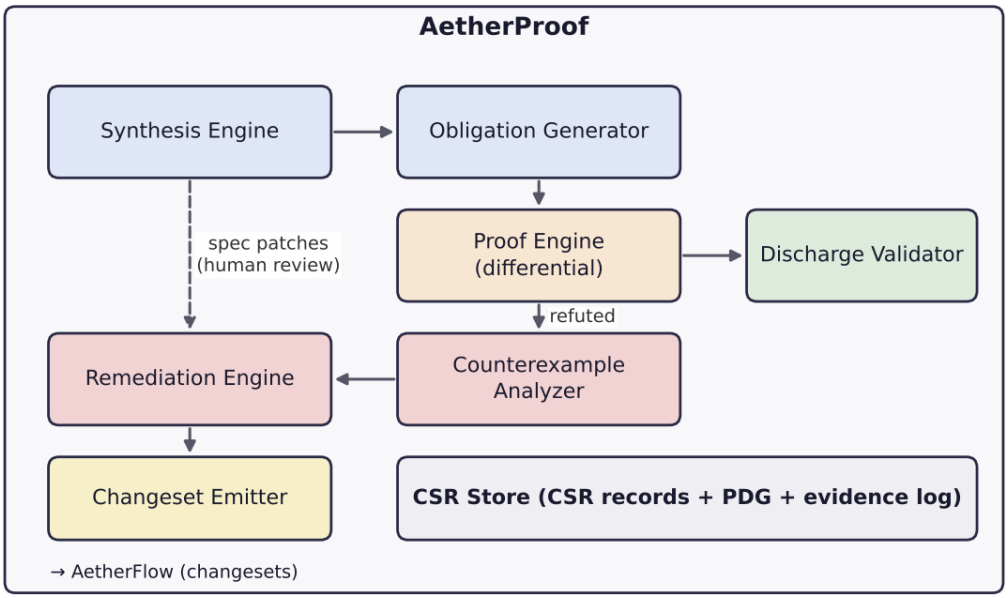


Figure 13. AetherProof component map. Remediation patches leave only through the Changeset Emitter, into AetherFlow; specification patches are routed to human review.

Component	Responsibility	Input	Output
Synthesis Engine	Generate candidate specifications from requirements and code	Requirements, changesets	Candidate CSR records
Obligation Generator	Generate proof obligations from CSR records	CSR records	Obligations (Section 4)
Proof Engine	Discharge obligations by differential proving	Obligations, evidence	Proof evidence (Section 5)
Discharge Validator	Validate evidence and TCB manifests	Evidence, manifests	Validated evidence
Counterexample Analyzer	Diagnose counterexamples	Refuted obligations	Diagnosed counterexamples (Section 6)
Remediation Engine	Propose and validate patches	Counterexamples	Remediation changesets
Changeset Emitter	Submit patches to AetherFlow	Remediation patches	Changesets via the Section 7 protocol
CSR Store	Persist CSR records, PDG, and evidence	All of the above	Persistent state

The Synthesis Engine is the component most exposed to threat T15: candidate specifications it produces enter the CSR only through human review, and the architecture does not substitute language-model-generated proofs for independently checkable evidence.

10.3 Data Flows

Requirement-to-evidence flow. Requirement → [Synthesis Engine] → candidate specification → [human review] → CSR record → [Obligation Generator] → proof obligation → [Proof Engine] → proof evidence + TCB manifest → [Discharge Validator] → verified evidence → [CSR store] → persistent evidence.

Counterexample flow. Refuted obligation → [Counterexample Analyzer] → diagnosed counterexample → [Remediation Engine] → patch proposal → [Guardrails] → accepted or rejected patch → [Changeset Emitter] → changeset to AetherFlow.

10.4 Interfaces

Interface	Direction	Protocol
AetherProof ↔ AetherFlow	Bidirectional	Changeset and counterexample messages (Section 7)
AetherProof ↔ AetherGit	Read-only	Changeset DAG via the AetherGit API
AetherProof ↔ Human	Bidirectional	CSR review, escalation (Section 8)
AetherProof ↔ External tools	Outbound	TLA+, Why3, Lean, Z3 tool interfaces

Internal interfaces are defined by the CSR store schema and the PDG structure and are not exposed externally.

10.5 Non-Goals and Limitations

Non-goals. AetherProof does not aim to: (1) unify all specification languages into one; (2) replace human judgment in specification authoring or remediation; (3) claim absolute correctness of any system; (4) substitute LLM-generated proofs for independently checkable evidence; (5) assume semantic consistency without proof; (6) treat layer position as semantic refinement; (7) silence counterexamples by silently modifying specifications; (8) guarantee full automation of remediation.

Architectural constraints. No implementation is benchmarked, so the architecture is descriptive, not empirical; there is no real-world deployment, so performance, scalability, and usability are unknown; there is no empirical validation, so the claims are structural; and integration is protocol-only, so cross-system performance is unknown. **Known unknowns.** The computational cost of differential proving at scale; the number of obligations that can be managed; the burden on human reviewers; and whether soundness is preserved under concurrent agent activity.

10.6 Illustrative End-to-End Walk-Through

> ***Status:*** *this walk-through is hypothetical. The specification fragments are written to illustrate structure;* ***none has been executed with TLC, Why3, Lean, or Z3****, and no verdict below is the output of a tool. Statements that a formula is unsatisfiable follow from elementary integer reasoning, not from a solver run.*

Requirement and specifications

Requirement R1: *the agent work queue must never hold more than MAX items, where MAX is a positive constant.* It is formalized at three layers.

```
---- MODULE BoundedQueue ----          \* L1 (TLA+), csr-L1-001
EXTENDS Naturals
CONSTANT MAX
ASSUME MAX \in Nat /\ MAX > 0       \* assumption A1
VARIABLE queue_length
Init    == queue_length = 0
Enqueue == queue_length < MAX /\ queue_length' = queue_length + 1
Dequeue == queue_length > 0   /\ queue_length' = queue_length - 1
Next    == Enqueue \/ Dequeue
```

```
Inv     == queue_length <= MAX
Spec    == Init /\ [][Next]_queue_length
THEOREM Spec => []Inv
====

(* L2 (WhyML-style, illustrative), csr-L2-001, refines csr-L1-001 *)
let enqueue (q: queue) : queue
  requires { 0 <= q.len < max }
  ensures  { result.len = q.len + 1 /\ result.len <= max }
= { len = q.len + 1 }

; L3 (SMT-LIB), csr-L3-001, instantiates the arithmetic content of csr-L2-001
(declare-const len Int) (declare-const mx Int)
(assert (> mx 0))
(assert (and (<= 0 len) (< len mx)))
(assert (not (<= (+ len 1) mx)))   ; negation of the postcondition bound
(check-sat)                        ; unsatisfiable by integer arithmetic
```

Generated obligations and ODG

The Obligation Generator produces O-001 (Invariance, from csr-L1-001, assumption A1), O-002 (PrePost, from csr-L2-001), and O-003 (Satisfiability, from csr-L3-001). A plausible ODG records that O-001 `requires` O-002 and O-002 `requires` O-003. Each obligation is initially `Unknown`. O-003 is dispatched to Tier 1 (Z3); O-002 to Why3; O-001 to TLC for a bounded instance and, if unbounded assurance is required, TLAPS. Each discharge produces evidence with its per-path manifest.

A changeset introduces a defect

An agent submits changeset c_k that changes the guard of `enqueue` from `q.len < max` to `q.len <= max`. AetherFlow admits it without blocking and sends M1. AetherProof identifies that the source CSR record's implementation falls inside the blast radius of O-002 and O-001, invalidates them (trigger 4 of Section 5.3), and tests reuse: the assumptions, property, and relations are unchanged, but the artifact the old evidence was computed against has changed, so the proposed artifact-stability condition fails (Section 5.4) and both are re-discharged. O-003 is unaffected and keeps its evidence.

Re-discharge of O-002 now fails. With the weakened guard, the precondition admits `q.len = max`, after which the result has length `max + 1`, violating the bound. The witness is `queue_length = MAX` before the call, leading to `MAX + 1` after. The verdict is `Refuted`; AetherProof sends M4.

Diagnosis and remediation

Diagnosis follows Section 6.3. The witness is reproducible (re-running gives the same witness). It does not falsify assumption A1 (MAX is still positive). It is a valid model of the actual implementation, because the changed guard really does admit that state. The specification (the bound) expresses requirement R1 correctly and is the thing violated, so the counterexample is classified as a **code defect**. The Remediation Engine proposes restoring the strict guard. The guardrails pass: no test or assertion is modified, test infrastructure is untouched, the patch lies within the blast radius, and the specification is not weakened. The patch is sent as M5 and admitted as a new changeset c_{k+1}. O-002 and O-001 are re-discharged on the patched code and return `Proven` with evidence and manifests; M3 carries the result to AetherFlow.

DOI: https://doi.org/10.5281/zenodo.24013124

Two rejected alternatives

Tempting patch	Guardrail / rule that stops it	Outcome
Relax the invariant to queue_length <= MAX + 1	Specification integrity; specification change requires human review (P7)	Escalated to a Reviewer, who must trace the change to R1; R1 says MAX, so the change is expected to be rejected
Add assumption A2: queue_length < MAX before every call	Assumption tracking; assumption change requires review; the added assumption would make the property hold vacuously	Rejected or escalated; dependents invalidated; reviewer asks whether A2 is true of the environment
Delete or disable the failing check	Test and assertion integrity	Rejected; cycle returns to diagnosis

The example illustrates the intended division of labor: the engines find and repair the defect quickly when it is a plain code defect, and the architecture routes anything that touches the *meaning* of the specification to a person. It also illustrates a limit: if R1 itself had been mis-formalized (T3), every proof above would still succeed.

Limitations of this section. This section is an architecture description plus one hypothetical illustration. Its non-goals are as important as its components, and no component has been implemented or evaluated; the scalability, cost, and failure behavior of the assembled system are open (Q25–Q30).

11. Discussion

11.1 What Formalization Adds

Each element of AetherProof has precedents: heterogeneous formal toolchains, assumption management in proof engineering, incremental re-verification, repair loops. What formalization adds is a vocabulary in which several informal practices can be stated precisely enough to be questioned: that a translation between specification languages has a *classification* and a *justification* (Definition 3); that evidence has a manifest and a trust level, and trust does not propagate by position; that reuse of a proof requires named conditions (Section 5.4); that a remediation agent's search space must exclude edits to the verifier's own inputs (Section 6.5). None of these is novel in isolation. Their combination into a single recorded, reviewable structure is the contribution, and its value is conditional on the open questions being answered.

11.2 The Semantic Consistency Challenge

The central commitment of the paper is that the hard problem is **not** making TLA+, Why3, Lean, and SMT-LIB exchange data; scripts can do that. It is establishing that *what the languages mean* remains consistent when a specification moves from one representation to another. A small example shows how easily meaning shifts. In Section 10.6, the TLA+ specification quantifies queue_length over the natural numbers; the Why3 contract uses mathematical integers; a production implementation may use fixed-width machine integers. An invariant proved over unbounded naturals does not carry over to an implementation where `len + 1` can overflow. Translating the L1 invariant into an L2 contract over mathematical integers is plausibly exact; translating it into a contract about a fixed-width implementation is a different relation — an under-approximation or a partial mapping — and the architecture's job is to force that difference to be written down rather than to let it pass unnoticed. This is what Axis A is for. Whether the difference can be *detected* mechanically, as opposed to recorded after a human notices it, is Q1, Q2, and Q7.

DOI: https://doi.org/10.5281/zenodo.24013124

11.3 The Human-in-the-Loop Necessity

There are three independent reasons why human review cannot be engineered away. First, **intent is not in the artifacts**: a specification can be consistent with the code and with every proof and still fail to express what was meant (T3). Second, **undecidability**: by Rice's theorem (Rice, 1953), no algorithm decides non-trivial semantic properties of programs in general, so some obligations will lie outside automation. Third, **authority**: decisions to change what a system is *supposed* to do are not verification decisions. Human review is also fallible and costly: the burden at scale is unknown (Q39), conflicts of interest are hard to detect (Q38), and ML-assisted triage (Q42) would introduce its own failure modes. The protocol does not promise reviewers are right; it promises that reviewed decisions are attributable, evidence-backed, and revisitable.

11.4 When Not to Use AetherProof

The architecture is unlikely to be appropriate where requirements are unstable or unwritten, since specifications derived from code alone are bug-compatible; where the properties that matter are empirical (latency, usability) and not formal; where the toolchain cannot produce checkable evidence for the relevant properties; where change volume is low enough that conventional review and testing suffice; or where the cost of maintaining specifications and manifests exceeds the cost of the failures they would prevent. Formal verification has succeeded at scale mainly where the cost of failure is high and the component is stable and well specified (Klein et al., 2009; Leroy, 2009); whether it can be applied at the cadence of agent-generated change is the empirical question this architecture exists to make askable.

11.5 Relation to Prior Papers in the Series

AetherGit and AetherFlow each stated the open question their layer leaves for the next. AetherFlow noted that textual and CI evidence cannot detect conflicts invisible to tests; AetherProof is a proposal for what evidence beyond tests might look like. The papers are independent, and a reader may adopt any one without the others.

Limitations of this section. The discussion argues for the architecture's design choices; it does not demonstrate that they are the best choices. The example in Section 11.2 is illustrative and does not assess how common such mismatches are in practice.

12. Conclusion and Future Work

AetherProof is a formal architecture for specification-driven proof construction, incremental differential proving, and counterexample-guided remediation in multi-agent software ecosystems. Its principal structural commitments are that specification layers are functional categories related by recorded relations rather than unified or ranked; that a specification record has three distinct identities and is not a proof; that translations are classified along two orthogonal axes; that every obligation names its assumptions and every piece of evidence names its trusted base; that proof reuse requires stated conditions; that remediation is bounded, guarded, and subject to human authority over specifications; and that integration with the integration layer is by versioned, asynchronous, authenticated messages.

What has been established is a specification. What has not: no implementation, no benchmark, no empirical evaluation, no formal proof of P1–P10, and no resolution of the fifty open questions, among which semantic consistency (Q2) is the most consequential. The order of work is specify, validate, implement, evaluate.

DOI: https://doi.org/10.5281/zenodo.24013124

12.1 Future Work

Stage	Step	Deliverable
Immediate	Resolve Q1 (translation soundness) and Q2 (definition of semantic consistency)	Formal proof or counterexample; mathematical definition
Immediate	Specify CSR, PDG, and protocol schemas (JSON/YAML, JSON Schema)	Schema files
Immediate	Implement the CSR Store and the Obligation Generator	Reference implementations
Medium-term	Implement the Proof Engine, Discharge Validator, Counterexample Analyzer, Remediation Engine, and integration protocol	Reference implementations
Medium-term	Execute the empirical evaluation, with threats to validity (Section 9.9) addressed	Evaluation report (v2.0.0 or later)
Long-term	Extend to distributed systems and to empirical properties; study reviewer usability at scale	Architecture extensions; empirical study
Long-term	Submit to a peer-reviewed venue	Submission

A first empirical question that would be tractable early: for a small repository with a stated invariant, how often do plausible agent-generated changes that pass the existing tests violate the invariant, and how does the cost of differential re-proving compare with full re-proving? This paper does not answer that question and offers no prediction of its answer.

Limitations of this section. Future work is a plan, not a result; the ordering reflects dependencies among open questions, not commitments or schedules.

Appendix A — Notation

Symbol / term	Meaning
L1, L2, L3	Functional layers: system-level properties (TLA+), functional contracts (Why3/Lean), logical constraints (SMT-LIB/Z3)
csr, csr_id, artifact_hash, semantic_id	CSR record and its record, artifact, and semantic identities
refines, instantiates, approximates, implies, contradicts, depends_on, supersedes	The seven semantic relations
S, T; B(S), B(T)	Source and target of a translation; behavior or model sets they admit
Axis A	Semantic relationship: Exact, Over-approximation, Under-approximation, Partial mapping, Divergent
Axis B	Verification property relative to a named property: Sound, Complete, Neither, Not established
O = (id, property, assumptions, source_csr, discharged_by, verdict)	Proof obligation
ODG, PDG	Obligation Dependency Graph; Proof Dependency Graph
A = (id, statement, source, verified_by)	Assumption
C = (id, obligation, witness, refutation_evidence, type, provenance)	Counterexample
TCB	Trusted computing base; TCB manifest is its explicit record
Verdict	Proven / Refuted / Unknown
Disposition	Completed / Remediation Required / Further Analysis / Escalated to Human
δ (written d where a plain letter is required)	Content delta of a changeset
$c_i = (\delta_i, P(c_i), \tau_i, \sigma_i)$	AetherGit changeset tuple (Baladi, 2026b)
Q1–Q50, D1–D52, T1–T15, P1–P10	Open questions, design decisions, threats, propositions
M1–M10	Integration protocol messages

Appendix B — Open Questions Q1–Q50

All fifty questions are open. They are grouped by the source design document in which they arose (Arch. = system architecture document).

ID	Question	Source
Q1	Under what conditions is a translation sound with respect to a property?	01
Q2	What is a mathematically precise definition of semantic consistency?	01
Q3	What is the minimal CSR structure that preserves semantic identity?	01
Q4	How should assumption dependency graphs be structured?	01
Q5	When is an approximation “good enough” for a requirement?	01
Q6	How should proof portability interact with tool diversity?	01
Q7	Can the three identities of a CSR record be reconciled mechanically?	01
Q8	What is the minimal obligation structure that preserves traceability?	02
Q9	Under what conditions can an obligation be safely re-discharged after an assumption change?	02
Q10	How should the ODG be structured to support efficient incremental invalidation?	02
Q11	When is a bounded model-checking result of Unknown acceptable as a discharge?	02
Q12	How should human review be encoded as discharge evidence?	02

ID	Question	Source
Q13	What is the minimal PDG structure that supports sound invalidation and reuse?	03
Q14	Under what conditions can evidence be reused across changesets without re-discharge?	03
Q15	How should cost be estimated for complex obligations?	03
Q16	When should the engine escalate to a higher tier versus accept Unknown?	03
Q17	How should conflicting evidence be resolved?	03
Q18	Can re-validation be made incremental?	03
Q19	What is the minimal diagnosis protocol that distinguishes code defects from specification errors?	04
Q20	When is a counterexample “reproducible enough” to be actionable?	04
Q21	How should conflicting counterexamples be resolved?	04
Q22	What is the maximum iteration bound that preserves soundness?	04
Q23	How should guardrails be formalized to prevent specification gaming?	04
Q24	Can the remediation cycle be made confluent (order-independent)?	04
Q25	What is the minimal viable AetherProof implementation that demonstrates the architecture?	Arch.
Q26	How does AetherProof perform at scale (thousands of obligations, hundreds of changesets per day)?	Arch.
Q27	What is the human review burden per changeset?	Arch.
Q28	Can AetherProof integrate with existing CI/CD systems without modification?	Arch.
Q29	How does AetherProof handle heterogeneous toolchains (TLA+, Why3, Lean, Z3)?	Arch.
Q30	What is the failure mode when the synthesis engine produces incorrect specifications?	Arch.
Q31	What is the minimum protocol version that supports all needed messages?	05
Q32	How should backpressure be communicated between systems?	05
Q33	What is the maximum message size that both systems support?	05
Q34	How should audit logs be merged when the systems are temporarily disconnected?	05
Q35	Can the protocol be extended to support multiple AetherProof instances?	05
Q36	What is the latency SLA for each message type?	05
Q37	What is the minimal set of roles that preserves separation of duties?	06
Q38	How should reviewer conflicts of interest be detected and handled?	06
Q39	What is the cognitive burden of reviewing a specification change at scale?	06
Q40	Can human review be made auditable without slowing down the process?	06
Q41	How should human review evidence be re-validated after a specification change?	06
Q42	Can human review be partially automated (for example, by ML-assisted triage)?	06
Q43	Can tool signing be implemented without breaking existing workflows?	07
Q44	Can conflict-of-interest disclosure be automated?	07
Q45	Can blind review be implemented without losing traceability?	07
Q46	How should the architecture handle toolchain evolution?	07
Q47	How should Unknown obligations be prioritized for human review?	07
Q48	What is the maximum acceptable latency for human review?	07
Q49	Can the architecture be extended to distributed systems?	07
Q50	Can the architecture be extended to empirical properties (performance)?	07

Appendix C — Design Decisions D1–D52

ID	Decision
D1	Three functional layers (not hierarchical)
D2	CSR as semantic index
D3	Two-axis translation classification
D4	Traceability chain from requirements to proof
D5	Separate verification verdict from workflow disposition
D6	Per-path TCB manifest
D7	Three CSR identities
D8	Contradiction is a finding, not an escalation
D9	Obligations are tuples with explicit assumptions
D10	ODG is a DAG
D11	Assumption changes invalidate dependent obligations
D12	Discharge requires a TCB manifest
D13	Verdict and disposition are orthogonal
D14	Failure to discharge is recorded, not hidden
D15	PDG extends ODG with evidence and assumptions
D16	Invalidation propagates via derives_from edges
D17	Reuse requires assumption preservation and evidence compatibility
D18	Tiered proving with escalation
D19	Concurrency model is decentralized and stateless
D20	Audit log is append-only and checked in
D21	Re-validation is periodic, not continuous
D22	Counterexample type is diagnosed, not assumed
D23	Remediation cycle is bounded
D24	Specification changes are explicitly tagged
D25	Guardrails are mandatory
D26	Re-verification requires code and specification
D27	Escalation is a valid outcome
D28	Remediation is a changeset
D29	Three-layer architecture, protocol-based integration
D30	CSR store as central component
D31	Changeset Emitter routes all patches through AetherFlow
D32	Human review is a first-class component
D33	Non-goals are explicit
D34	Limitations are stated
D35	Protocol-based integration, not code-based
D36	Asynchronous communication
D37	Versioned protocol

ID	Decision
D38	Bounded queues
D39	Signed messages
D40	Shared audit log
D41	Human review as a first-class message
D42	Five roles defined, separation of duties enforced
D43	Automatic escalation for specification changes
D44	Required evidence per review type
D45	Decision Records are append-only and signed
D46	Human review as discharge evidence
D47	Trust is explicit, not implicit
D48	Threats are explicitly catalogued
D49	Limitations are explicitly stated
D50	Residual risk is acknowledged
D51	Threats to validity are catalogued
D52	Open questions are listed

Appendix D — Threats T1–T15

ID	Threat	Category	Severity	Mitigation	Residual risk
T1	Unsound proof artifact	Soundness	High	Independent checking	Low (if checker is trusted)
T2	Incomplete TCB manifest	Trust	High	Per-path TCB	Medium
T3	Specification does not reflect intent	Specification	High	Human review	Medium
T4	Silent assumption change	Trust	High	Assumption tracking	Low
T5	Guardrail bypass	Process	High	Mandatory guardrails	Low
T6	Toolchain compromise	Toolchain	High	Signed tools, TCB	Medium
T7	Reviewer conflict of interest	Human	Medium	Separation of duties	Low
T8	Incomplete completeness proof	Completeness	Medium	Explicit Unknown verdict	Medium
T9	Semantic inconsistency between layers	Trust	Medium	Explicit translation classification	Medium
T10	Counterexample flakiness	Soundness	Medium	Reproducibility check	Low
T11	Model over-approximation	Completeness	Medium	Model validation	Medium
T12	Hypothesis-testing bias	Human	Medium	Blind review	Medium
T13	Denial of service on proof engine	Process	Low	Bounded queues	Low
T14	Audit-log tampering	Process	High	Append-only, signed	Low
T15	LLM-generated specification error	Specification	High	Human review	Medium

Ratings are those recorded in the design documents; they are qualitative and unvalidated. Several mitigations (signed tools, blind review, multiple reviewers, conflict-of-interest disclosure, rate limiting, hash-chained logs, multiple language models) are named but not yet specified (Section 9.8).

References

1. Ananthanarayanan, S., Ardekani, M. S., Haenikel, D., et al. (2019). **Keeping Master Green at Scale.** Proceedings of the Fourteenth EuroSys Conference. https://doi.org/10.1145/3302424.3303970
2. Anthropic. (2026). **Redesigning Source Control at Anthropic.** Anthropic Research. https://www.anthropic.com/research/redesigning-source-control
3. Baladi, S. (2026a). **Rethinking Source Control for Agentic Software Engineering: Architectural Transformations, Data Models, and Scalability Constraints in AI-Driven Development.** Zenodo. https://doi.org/10.5281/zenodo.23934465
4. Baladi, S. (2026b). **AetherGit: Scalable Source Control for Autonomous AI Agents.** Zenodo. https://doi.org/10.5281/zenodo.23966896
5. Baladi, S. (2026c). **AetherFlow: Automated Post-Merge Verification and Dynamic Conflict Resolution in Multi-Agent Programming Environments.** Zenodo. https://doi.org/10.5281/zenodo.23979129
6. Barrett, C., Stump, A., & Tinelli, C. (2010). **The SMT-LIB Standard: Version 2.0.** Proceedings of the 8th International Workshop on Satisfiability Modulo Theories.
7. Chaudhuri, K., Doligez, D., Lamport, L., & Merz, S. (2010). **Verifying Safety Properties with the TLA+ Proof System.** IJCAR 2010, LNCS 6173, 142–148. https://doi.org/10.1007/978-3-642-14203-1_12
8. Clarke, E., Grumberg, O., Jha, S., Lu, Y., & Veith, H. (2000). **Counterexample-Guided Abstraction Refinement.** CAV 2000, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
9. Cousot, P., & Cousot, R. (1977). **Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.** POPL 1977, 238–252. https://doi.org/10.1145/512950.512973
10. de Moura, L., & Bjørner, N. (2008). **Z3: An Efficient SMT Solver.** TACAS 2008, LNCS 4963, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
11. de Moura, L., & Ullrich, S. (2021). **The Lean 4 Theorem Prover and Programming Language.** CADE-28, LNCS 12699, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37
12. Filliâtre, J.-C., & Paskevich, A. (2013). **Why3 — Where Programs Meet Provers.** ESOP 2013, LNCS 7792, 125–128. https://doi.org/10.1007/978-3-642-37036-6_8
13. Hoare, C. A. R. (1969). **An Axiomatic Basis for Computer Programming.** Communications of the ACM, 12(10), 576–580. https://doi.org/10.1145/363235.363259
14. Jimenez, C. E., Yang, J., Wettig, A., et al. (2024). **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?** ICLR 2024. arXiv:2310.06770.
15. Klein, G., Elphinstone, K., Heiser, G., et al. (2009). **seL4: Formal Verification of an OS Kernel.** SOSP 2009, 207–220. https://doi.org/10.1145/1629575.1629596
16. Klein, G., Andronick, J., Elphinstone, K., et al. (2014). **Comprehensive Formal Verification of an OS Microkernel.** ACM Transactions on Computer Systems, 32(1), Article 2. https://doi.org/10.1145/2560537
17. Lamport, L. (1994). **The Temporal Logic of Actions.** ACM Transactions on Programming Languages and Systems, 16(3), 872–923. https://doi.org/10.1145/177492.177726
18. Lamport, L. (2002). **Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.** Addison-Wesley.
19. Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). **GenProg: A Generic Method for Automatic Software Repair.** IEEE Transactions on Software Engineering, 38(1), 54–72. https://doi.org/10.1109/TSE.2011.104
20. Leino, K. R. M. (2010). **Dafny: An Automatic Program Verifier for Functional Correctness.** LPAR-16, LNCS 6355, 348–370.
21. Leroy, X. (2009). **Formal Verification of a Realistic Compiler.** Communications of the ACM, 52(7), 107–115. https://doi.org/10.1145/1538788.1538814
22. Luo, Q., Hariri, F., Eloussi, L., & Marinov, D. (2014). **An Empirical Analysis of Flaky Tests.** Proceedings of the 22nd ACM SIGSOFT FSE. https://doi.org/10.1145/2635868.2635920
23. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., & Deardeuff, M. (2015). **How Amazon Web Services Uses Formal Methods.** Communications of the ACM, 58(4), 66–73. https://doi.org/10.1145/2699417

24. Potvin, R., & Levenberg, J. (2016). **Why Google Stores Billions of Lines of Code in a Single Repository.** Communications of the ACM, 59(7), 78–87. https://doi.org/10.1145/2854146

25. Rice, H. G. (1953). **Classes of Recursively Enumerable Sets and Their Decision Problems.** Transactions of the American Mathematical Society, 74(2), 358–366.

26. Sapling SCM. (n.d.). **Sapling: A Scalable, User-Friendly Source Control System.** Meta Platforms. https://sapling-scm.com/

27. Thompson, K. (1984). **Reflections on Trusting Trust.** Communications of the ACM, 27(8), 761–763. https://doi.org/10.1145/358198.358210

Reference entries were assembled from the author's knowledge of the cited works and should be checked against publisher records before formal submission. References 2–5 are the sources for claims about Scion and the Aether family; claims about specific tools follow the cited publications.

Research Status Statement

Component	Current status
Three-layer functional specification architecture	Specified (Section 3)
Canonical Specification Representation; semantic relations; two-axis classification	Specified (Section 3)
Proof obligations and ODG; assumption tracking; discharge protocol	Specified (Section 4)
Incremental differential proof engine (PDG)	Specified (Section 5); invalidation algorithm and a fifth reuse condition proposed by this paper
Counterexample-guided remediation and guardrails	Specified (Section 6)
Integration protocol with AetherFlow	Specified (Section 7); mapping to AetherFlow's operator proposed, not agreed
Human review protocol and trust hierarchy	Specified (Section 8)
Threat catalogue (15 threats)	Catalogued with mitigations and residual risk (Section 9, Appendix D)
System architecture	Specified (Section 10)
Propositions P1–P10	Stated as design requirements; not formally proven
Illustrative walk-through	Hypothetical; no tool executed (Section 10.6)
Open questions (50) and design decisions (52)	Listed (Appendices B, C)
Implementation of any component	Not built
Benchmark / simulation / empirical evaluation	Not performed
Semantic consistency (Q2)	Open; provisional definition only

Reconciliation notes. The paper was assembled from eight numbered design documents and a system-architecture document. Where they differ, this paper chose as follows. (i) *Verdict range:* three values, with Escalated a disposition (Section 4.1). (ii) *Assumption match:* subset reading (Section 4.4). (iii) *Acyclicity and contradicts edges:* acyclicity applies to dependency-bearing edges (Section 4.3). (iv) *PDG node types:* five, adding changeset and CSR-record anchors to the three named (Section 5.2). (v) *Invalidation algorithm:* body completed by this paper (Section 5.3). (vi) *Reuse:* artifact-stability condition proposed (Section 5.4). (vii) *P9:* operational formulation (Section 8.7). (viii) *Series numbering:* this paper cites its siblings as Baladi (2026b) and (2026c); the status of any implementation of the sibling architectures is outside the claims of this paper.

Record ID: AETHERPROOF-01 · Version 1.0.0 · DOI: https://doi.org/10.5281/zenodo.24013124
Open questions: 50 · Design decisions: 52 · Threats: 15 · Propositions: 10 (design requirements) · Figures: 5 prescribed (Figures 1, 4–7) plus 7 additional.
Scope: formal architecture and specification paper. No implementation or evaluation reported. Next stage: v2.0.0 (implementation and evaluation).

Samir Baladi · Interdisciplinary AI Researcher · Ronin Institute / Rite of Renaissance · ORCID: 0009-0003-8903-0029
gitdeeper@gmail.com
© 2026 Samir Baladi